デザインとプログラミング 2026
かたちとコード / 基本図形と色彩による画面構成

慶應義塾大学環境情報学部
田所 淳

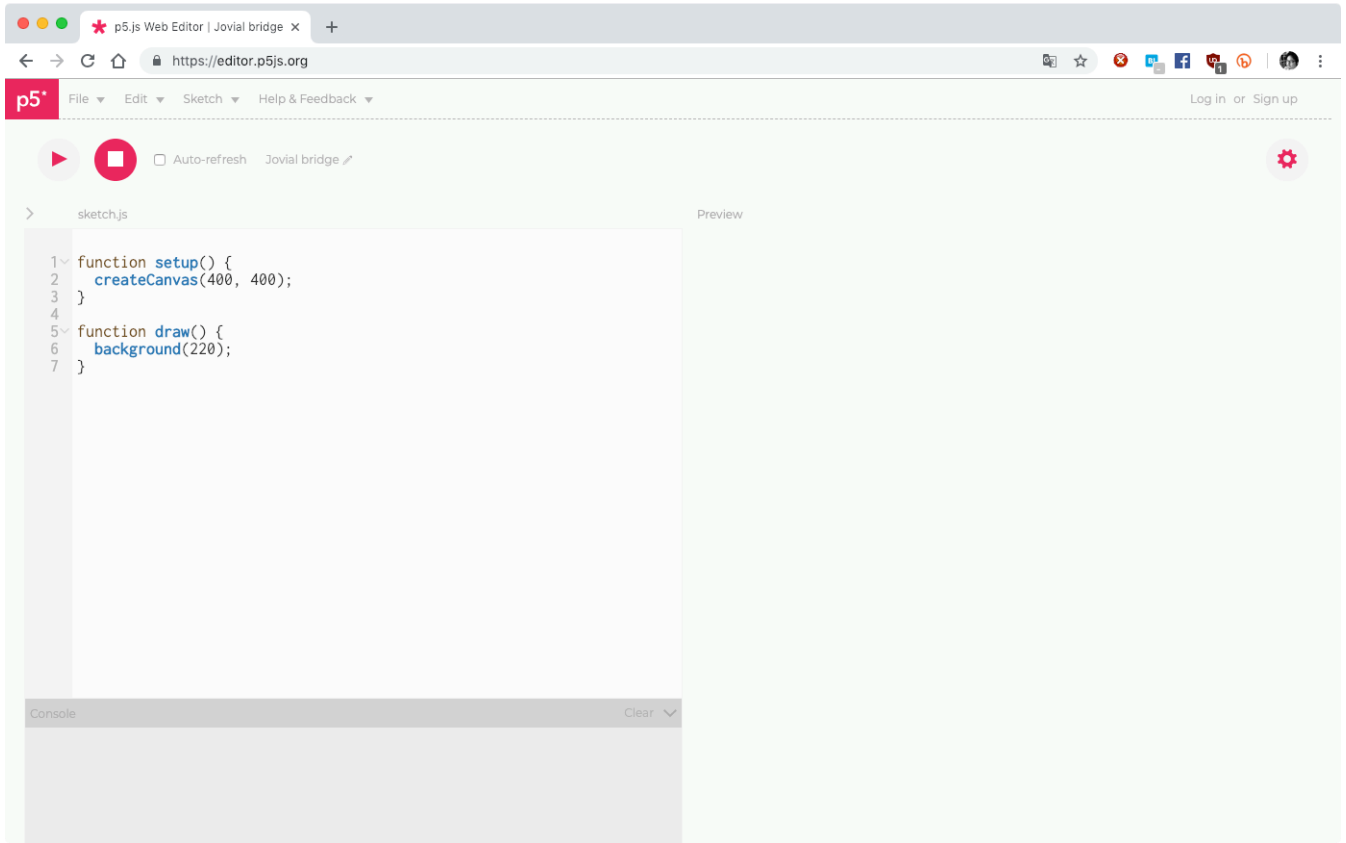
今日の内容

- 「かたちとコード」について
- p5.js入門
- 座標について
- 形を描いてみる
- 実習

p5.jsはじめの一歩

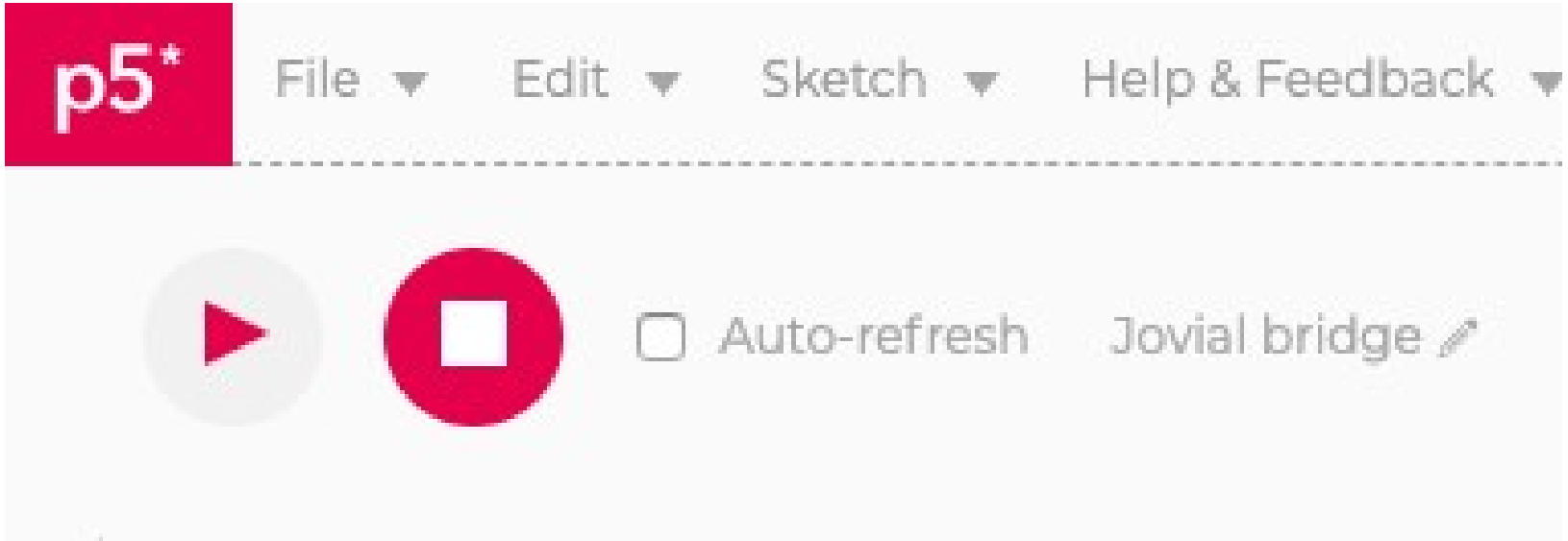
p5.jsはじめの一歩

- 一番簡単な方法は、p5.js Web Editorを使う方法 - 今日はこれで!
- https://editor.p5js.org/



p5.jsはじめの一歩

- 操作は簡単!!
  - 実行(Run)ボタン：プログラムの実行
  - 停止(Stop)ボタン：プログラムの停止



p5.jsはじめの一歩

- p5.jsを起動した段階で、既にプログラムのテンプレートが入っている!

```
function setup() {
  createCanvas(400, 400);
}

function draw() {
  background(220);
}
```

p5.jsはじめの一歩

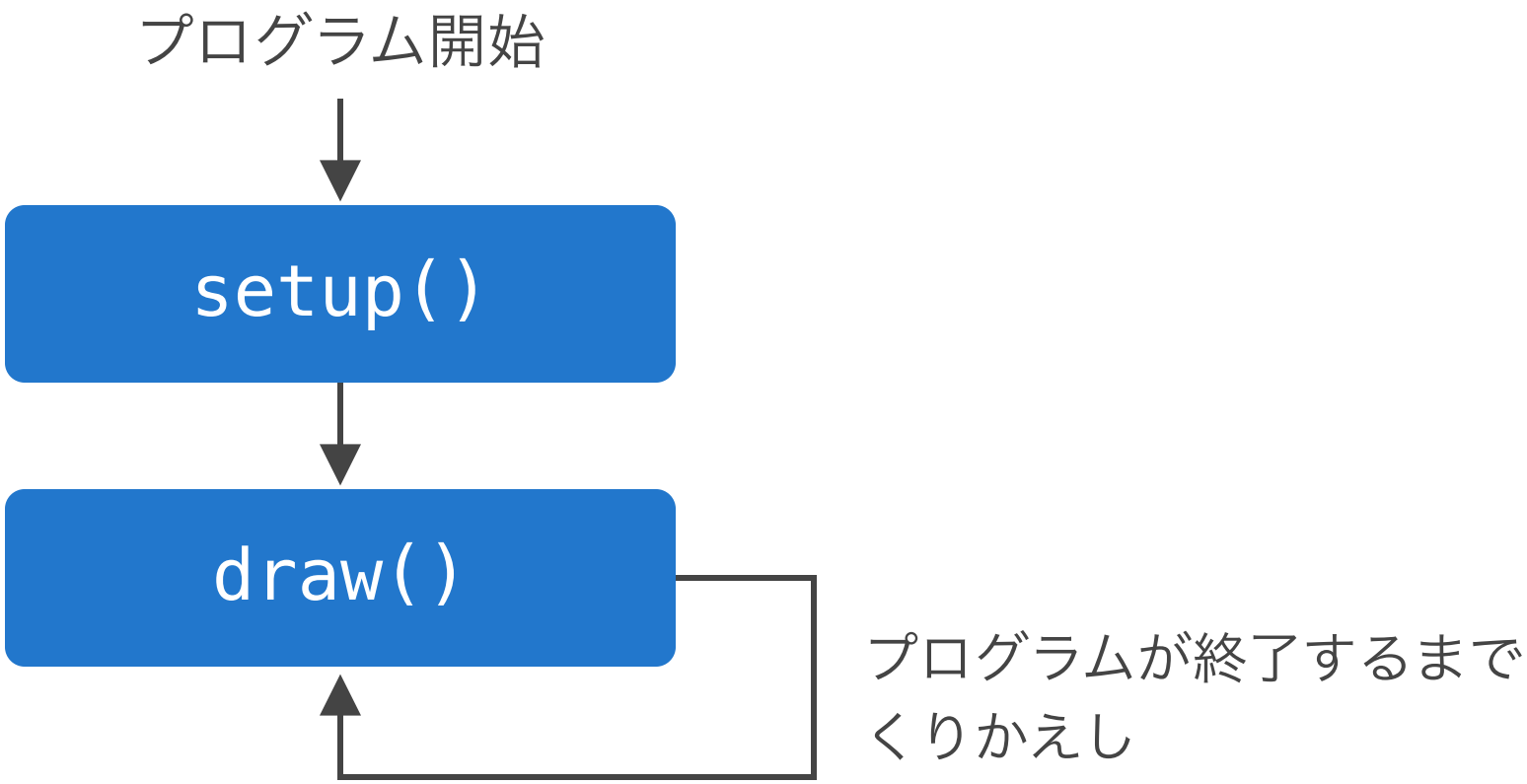
- function setup() と function draw() は何なのか?
- → アニメーションを実現する仕組み
- すこしずつ変化する画像を一定間隔で入れ替えている
- パラパラ漫画のイメージ

p5.jsはじめの一歩

- setup()とdraw()という二つのパートに構造化してアニメーションを実現
- **setup() - 初期設定:**
  - プログラムの起動時に一度だけ実行
  - 画面の基本設定やフレームレートなどを設定します。
- **draw() - 描画:**
  - 設定した速さ(フレームレート)でプログラムが終了するまでくりかえし実行されます。

p5.jsはじめの一歩

- setup()とdraw() のイメージ



p5.jsはじめの一歩

- setup：初期化関数
  - プログラムの最初に、1回だけ実行される処理を記述
  - アニメーションの前準備
- setupの中で行われることの多い処理
  - createCanvas：画面のサイズを設定
  - frameRate：画面の書き換え速度を設定
  - colorMode：カラーモードを設定

p5.jsはじめの一歩

- draw：メインループ関数
  - プログラムが終了するまでくりかえし
  - ループの中で図形の場所や色、形を操作してアニメーションにする
  - 画面の書き換え頻度はframeRate()関数で設定する

p5.jsはじめの一歩

- 実行の順序：上から順番に読みこまれていく
- 半角の英数字 (全角はダメ) のみを使用すること
- 文末にはセミコロン “;” を入れる
- いろいろな括弧が入れ子構造になっている
- おなじ括弧に囲まれている部分がひとつのブロック
- 最小単位 → 関数
    - 関数名(引数);

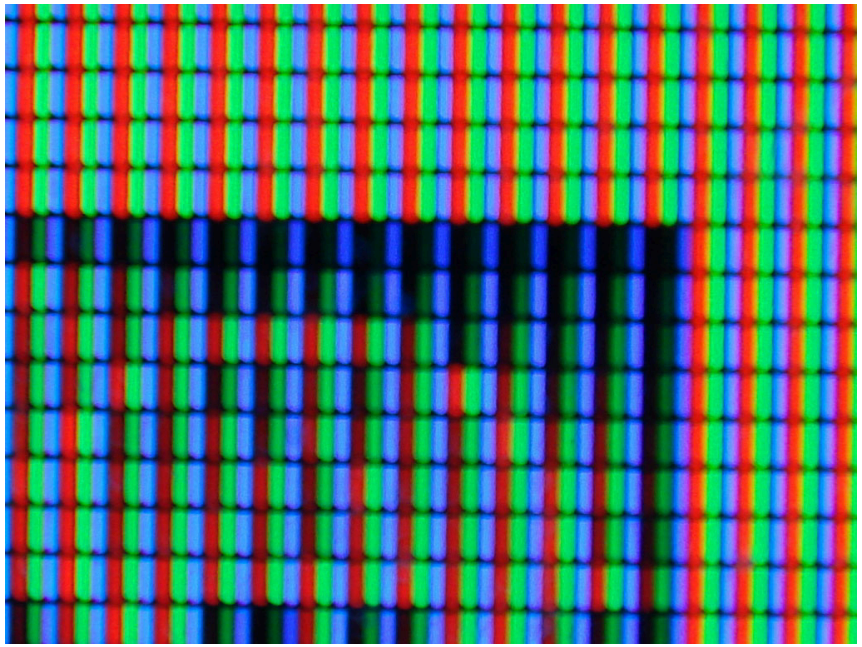
関数

- 関数 (function) とは
  - 引数と呼ばれるデータを受け取り、定められた通りの処理を実行して結果を返す一連の命令群。
- p5.js → ビジュアルプログラミングのための関数の集合
  - 関数名とその引数（パラメータ）から構成される
  - 引数の数は関数によって異なる

```
関数名(引数1, 引数2, 引数3...);
```

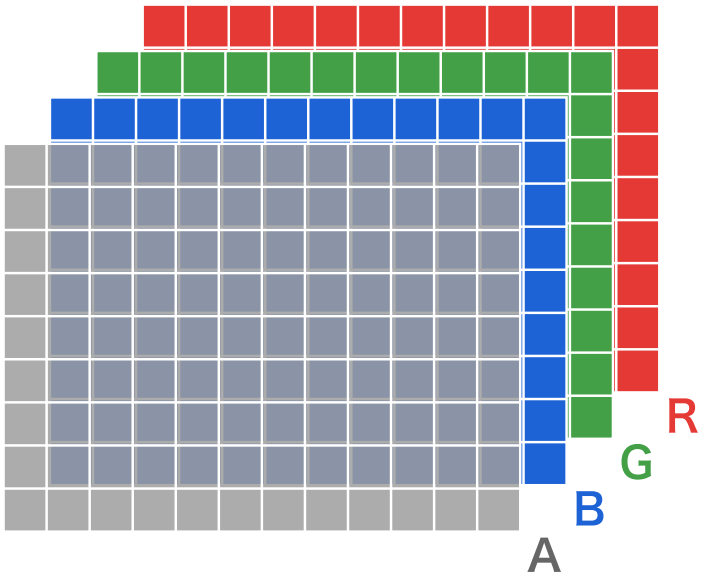
コンピュータで絵を描くということ

- コンピュータの画面はどうなっているのか?
  - コンピュータの画面を拡大していくと…
  - 縦横に並んだ点の集合 → ピクセル (Pixel)
  - 一つのピクセルは赤、緑、青の三原色から成り立っている



コンピュータで絵を描くということ

- コンピュータ画面は縦横沢山のピクセルから構成された巨大なエクセルの表のようなもの
- 例：1024 x 768 の液晶画面
  - 横に1024列縦に768行ならんだ巨大な表
  - それぞれのセルにR,G,B,A(アルファ値)が格納されている

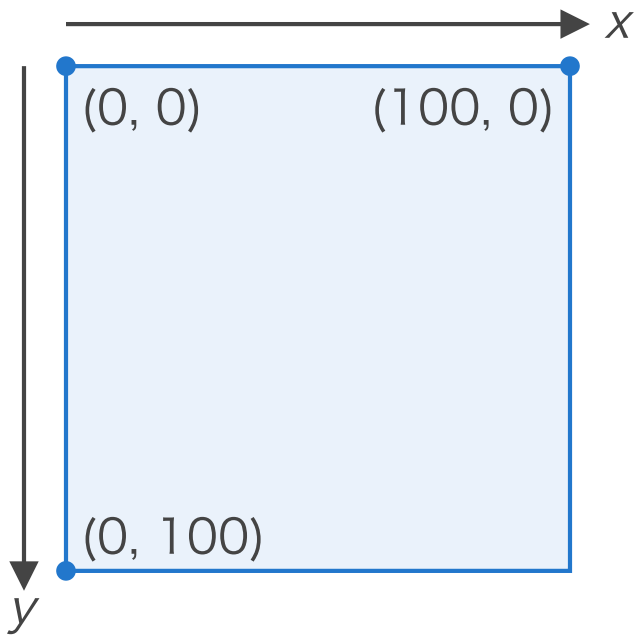


座標

- 座標 (Coordinate)：
  - 点の位置を明確にするために与えられる数の組のこと
- コンピュータの画面の1点を指定するためには、いくつのパラメータが必要か?
  - 2つの数字 (横と縦)： (x, y)
  - 2次元平面における直交座標

Processingの座標系

- 左上が原点 (0, 0)
- 右に行くほどx座標の値が増える
- 下に行くほどy座標の値が増える
- 例：100 x 100の平面の座標系



キャンバスを用意する

- 形を描いていく、画面 (キャンバス) を用意する
- createCanvas関数：キャンバスの大きさを指定

```
createCanvas(<幅>, <高さ>);
```

- 例：幅320pixel, 高さ240pixelのウィンドウを開く

```
createCanvas(320, 240);
```

描画画面の生成、コメントアウト

- プログラムにコメントを入れることで、分かり易く

```
//一行だけのコメント

/*
 複数の行にまたがる
 コメント
 */
```

点を描く

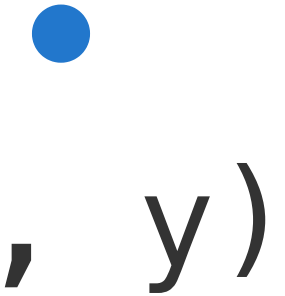
- point関数：点を描く

```
point(<X座標>, <Y座標>);
```

- 例：X座標100, Y座標120の位置に点を描く

```
point(100, 120);
```

点を描く



直線を描く

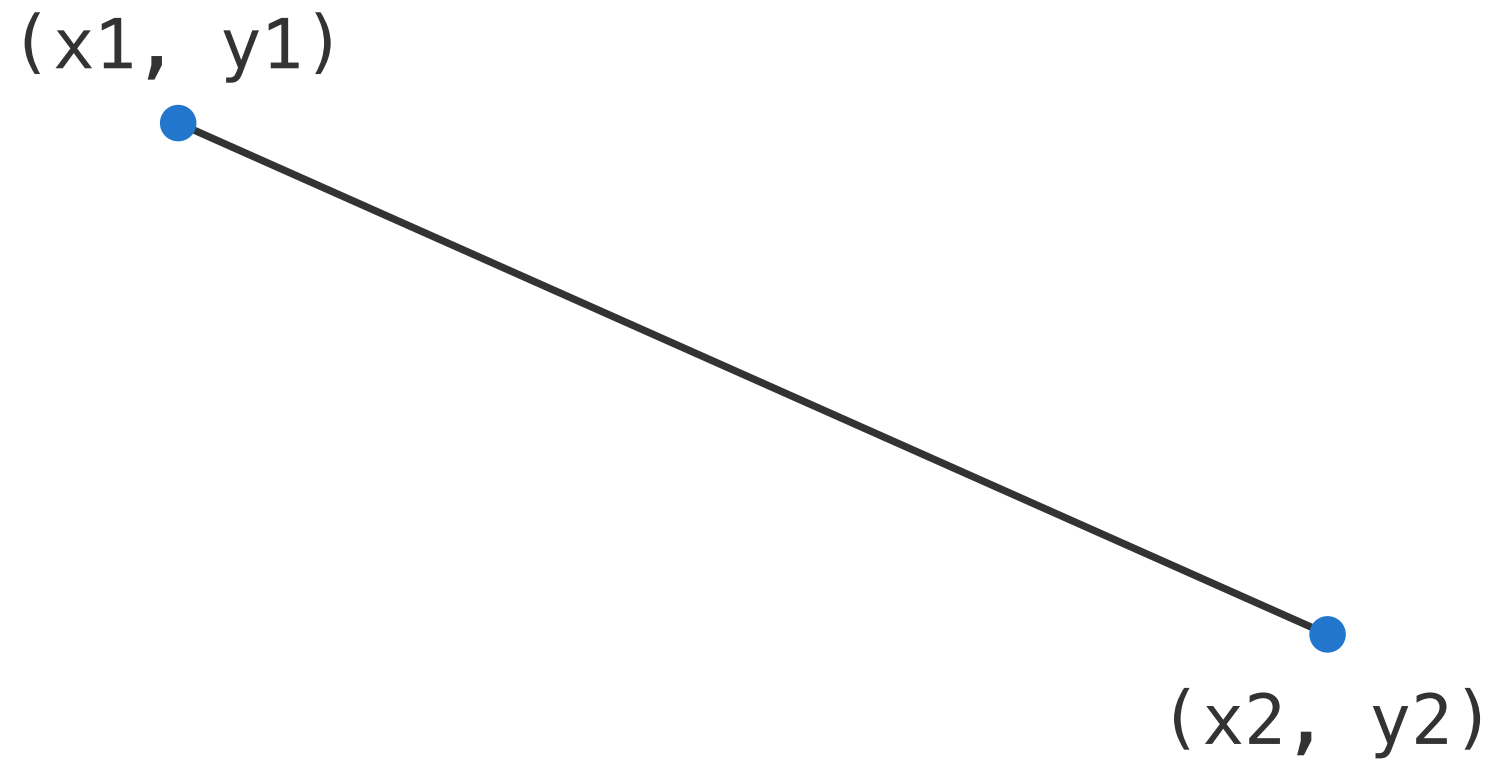
- line関数：直線を描く

```
line(<X座標始点>, <Y座標始点>, <X座標終点>, <Y座標終点>);
```

- 例：

```
line(10, 10, 200, 180);
```

直線を描く



長方形を描く

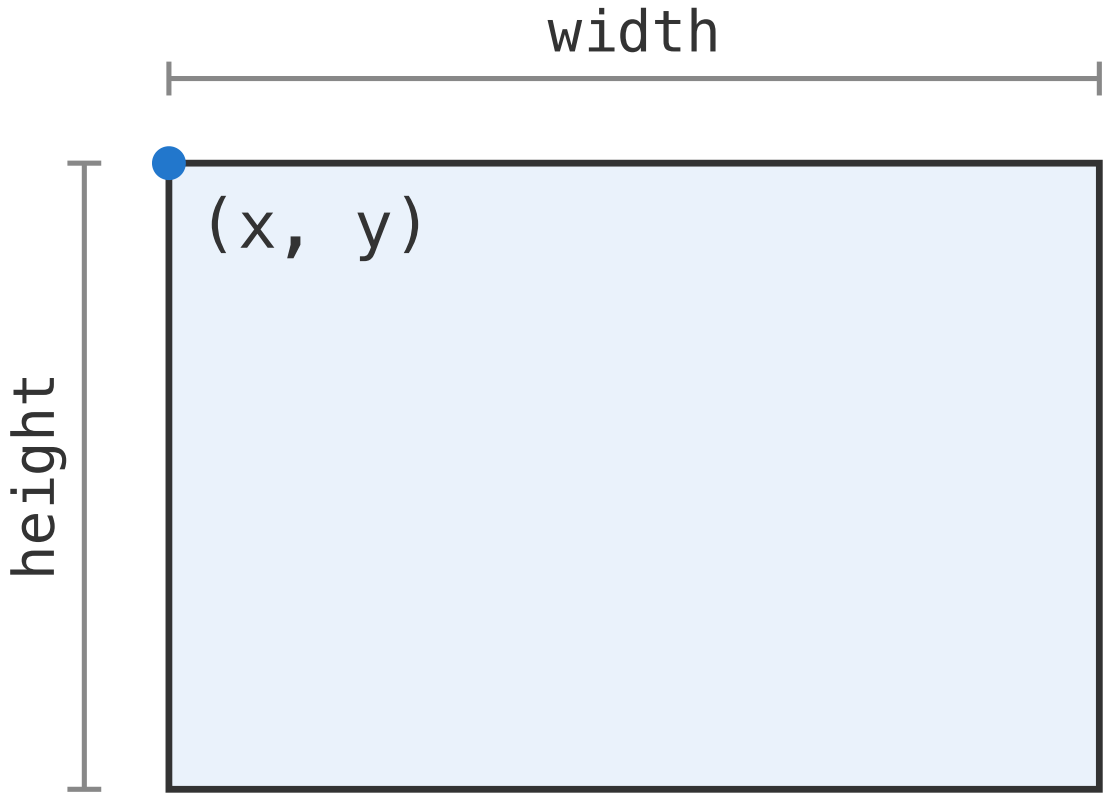
- rect関数：長方形を描く

```
rect(<X座標>, <Y座標>, <長方形の幅>, <長方形の高さ>);
```

- 例：

```
rect(10, 10, 200, 180);
```

長方形を描く



楕円を描く

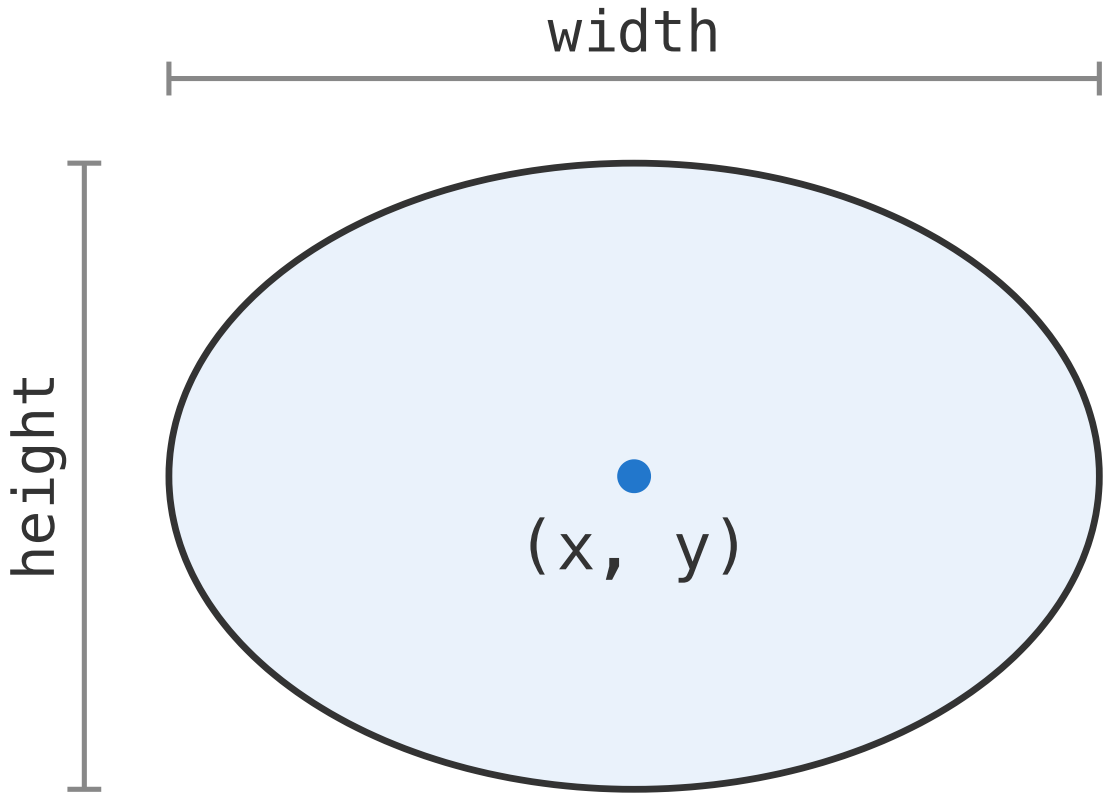
- ellipse関数：円、楕円を描く

```
ellipse(<X座標>, <Y座標>, <楕円の幅>, <楕円の高さ>);
```

- 例：

```
ellipse(10, 10, 200, 180);
```

楕円を描く



基本図形を描く

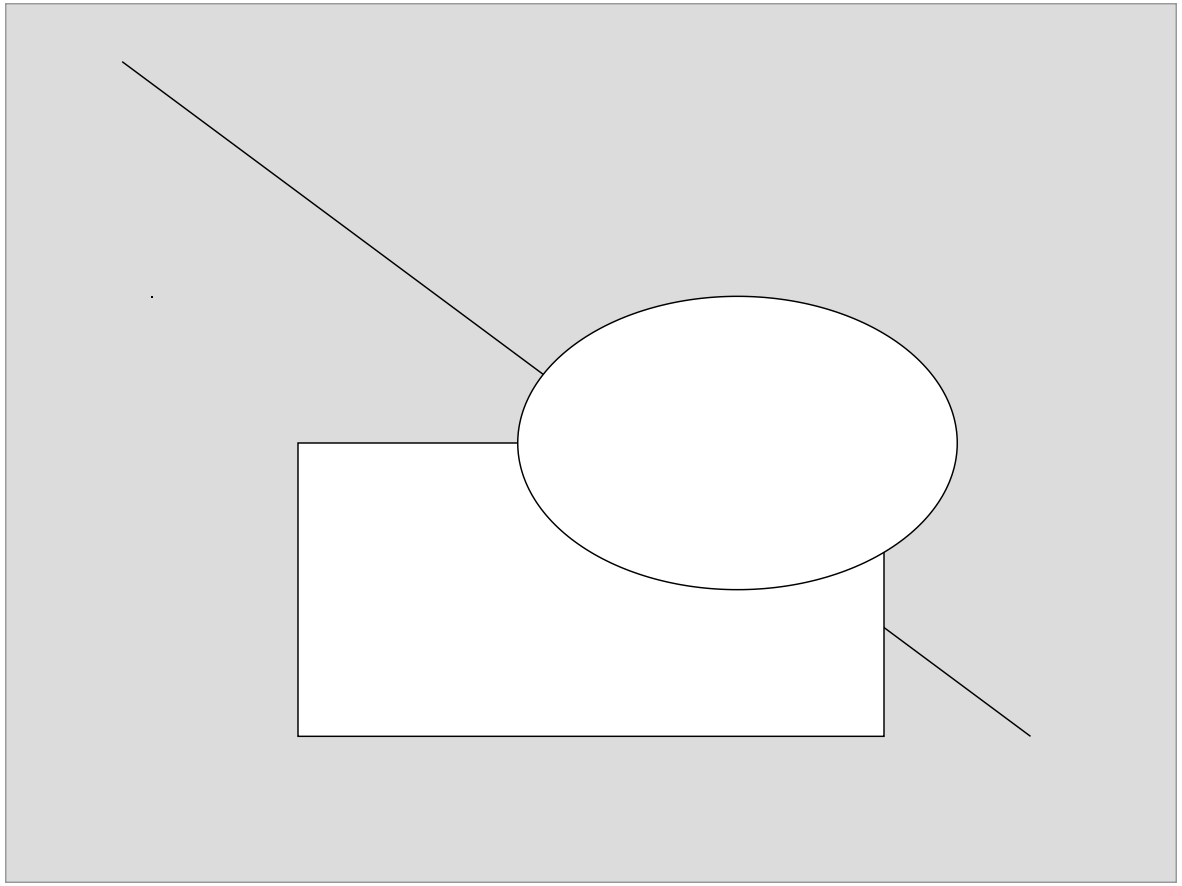
- ここまで出てきた基本図形を描いてみましょう

```
function setup() {
  createCanvas(800, 600);
}

function draw() {
  background(220);
  point(100, 200); // 点を描画
  line(80, 40, 700, 500); // 線を描画
  rect(200, 300, 400, 200); // 四角形を描画
  ellipse(500, 300, 300, 200); // 楕円を描画
}
```

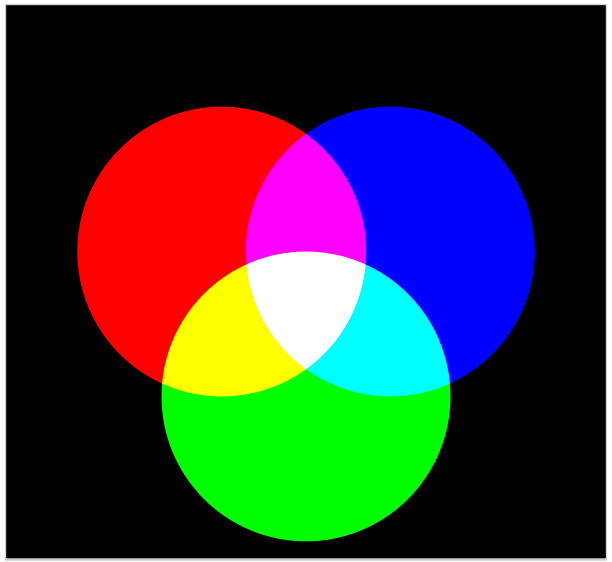
基本図形を描く

- できた!

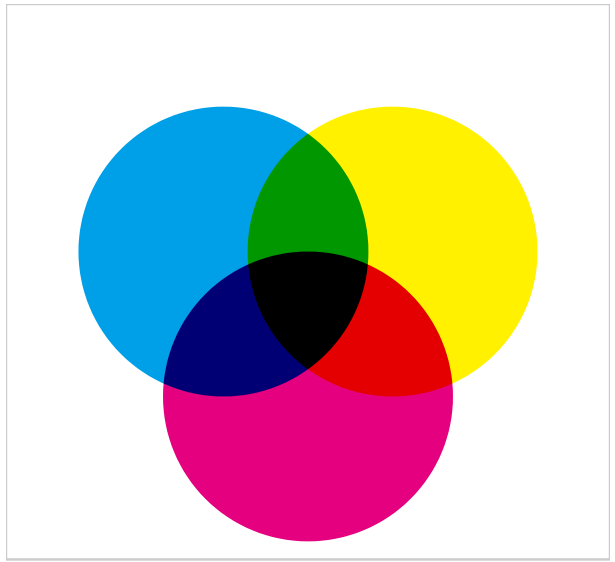


色の指定

- 色を指定するには?
  - R(赤) G(緑) B(青)の三原色で指定する
- 加法混色 (光の三原色であることに注意) ←→ 色料の三原色



光の三原色 (加法混色)



色料の三原色 (減法混色)

色の指定

- 3つの色の属性
- 背景色 background関数

```
background(<Rの値>, <Gの値>, <Bの値>);
```

- 線に色をつける stroke関数

```
stroke(<Rの値>, <Gの値>, <Bの値>);
```

- 塗りの色をつける fill関数

```
fill(<Rの値>, <Gの値>, <Bの値>);
```

Processingの色の塗りかたの規則

- 色や線塗りつぶしの設定は、それ以降すべての描画に使われる
- 色を変えるには改めて別の色を設定する命令を入れる

色の指定

- 背景色、塗りつぶしの色、ストロークの色の指定

```
//背景色
background(128); //グレースケールで指定
//塗りつぶしの色
fill(128, 64, 32); //RGB
//線の色
stroke(128, 64, 32); //RGB
```

色の指定

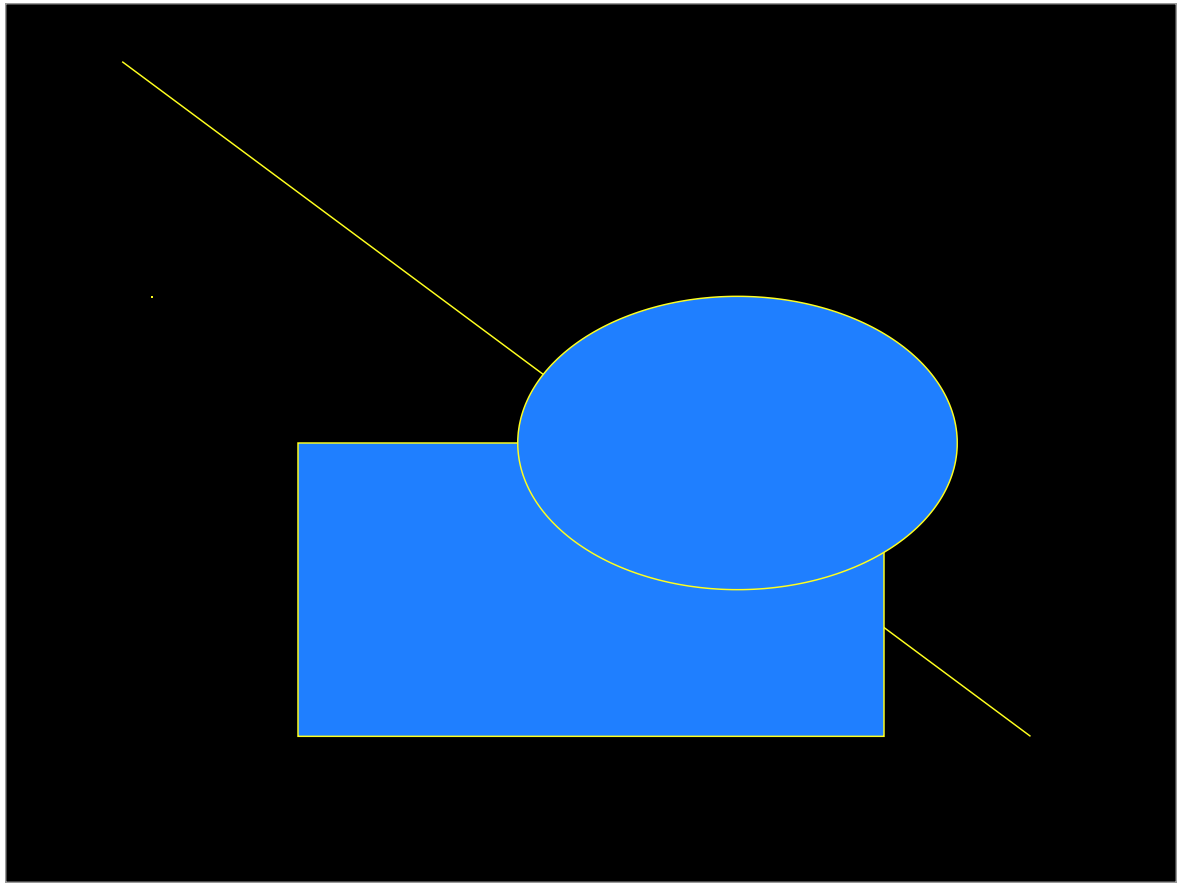
- さきほど描いた図形に色を塗ってみる

```
function setup() {
  createCanvas(800, 600);
}

function draw() {
  background(0); // 背景色
  stroke(255, 255, 31); //線の色
  fill(31, 127, 255); //塗り潰しの色
  point(100, 200); // 点を描画
  line(80, 40, 700, 500); // 線を描画
  rect(200, 300, 400, 200); // 四角形を描画
  ellipse(500, 300, 300, 200); // 楕円を描画
}
```

色の指定

- 線と塗りの色が反映される



透明度

- R, G, Bの3つの値に続けて、もう1つ値を指定すると、透明度を意味する
- 背景色

```
background(<Rの値>, <Gの値>, <Bの値>, <透明度>);
```

- 線

```
stroke(<Rの値>, <Gの値>, <Bの値>, <透明度>);
```

- 塗り

```
fill(<Rの値>, <Gの値>, <Bの値>, <透明度>);
```

透明度

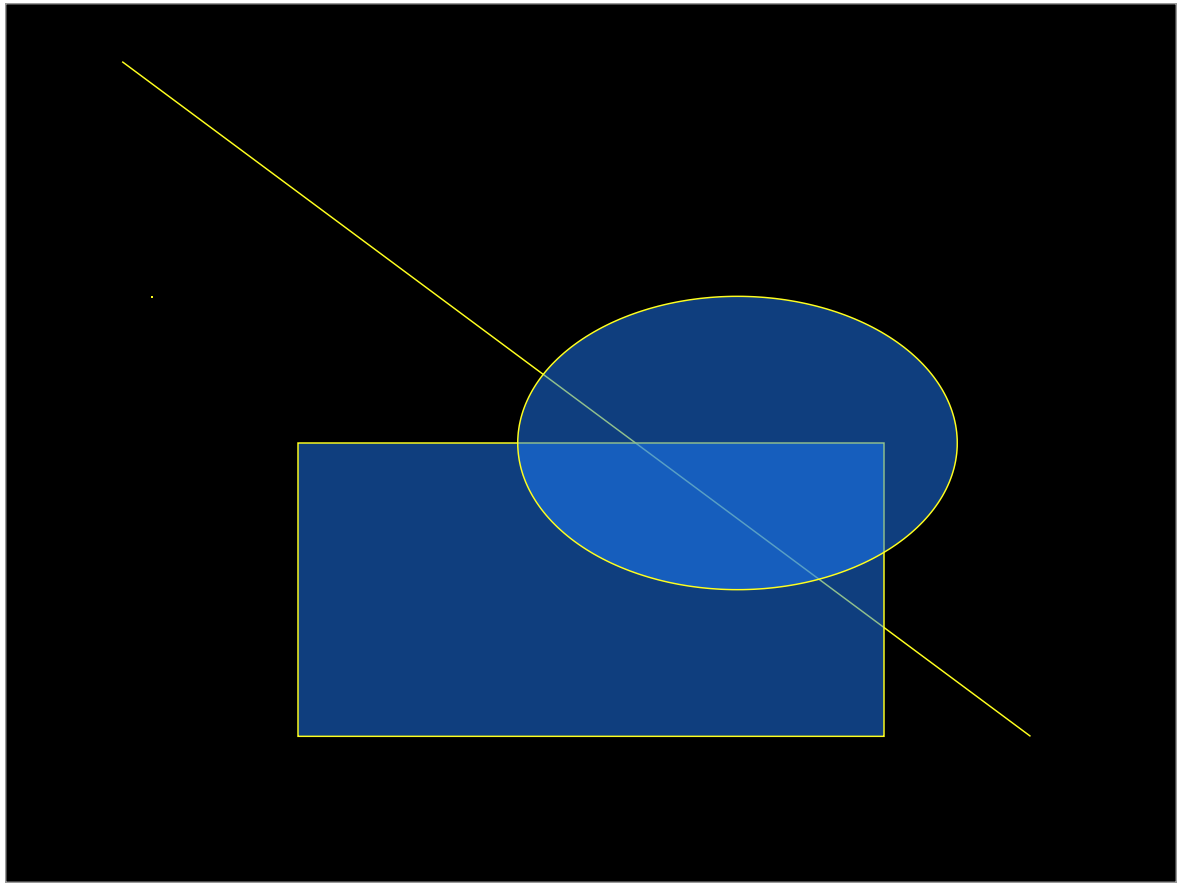
- 先程の例に透明度を付加してみる

```
function setup() {
  createCanvas(800, 600);
}

function draw() {
  background(0); // 背景色
  stroke(255, 255, 31); //線の色
  fill(31, 127, 255, 127); //塗り潰しの色(半透明)
  point(100, 200); // 点を描画
  line(80, 40, 700, 500); // 線を描画
  rect(200, 300, 400, 200); // 四角形を描画
  ellipse(500, 300, 300, 200); // 楕円を描画
}
```

透明度

- 透明度が付加される



色の塗り分け

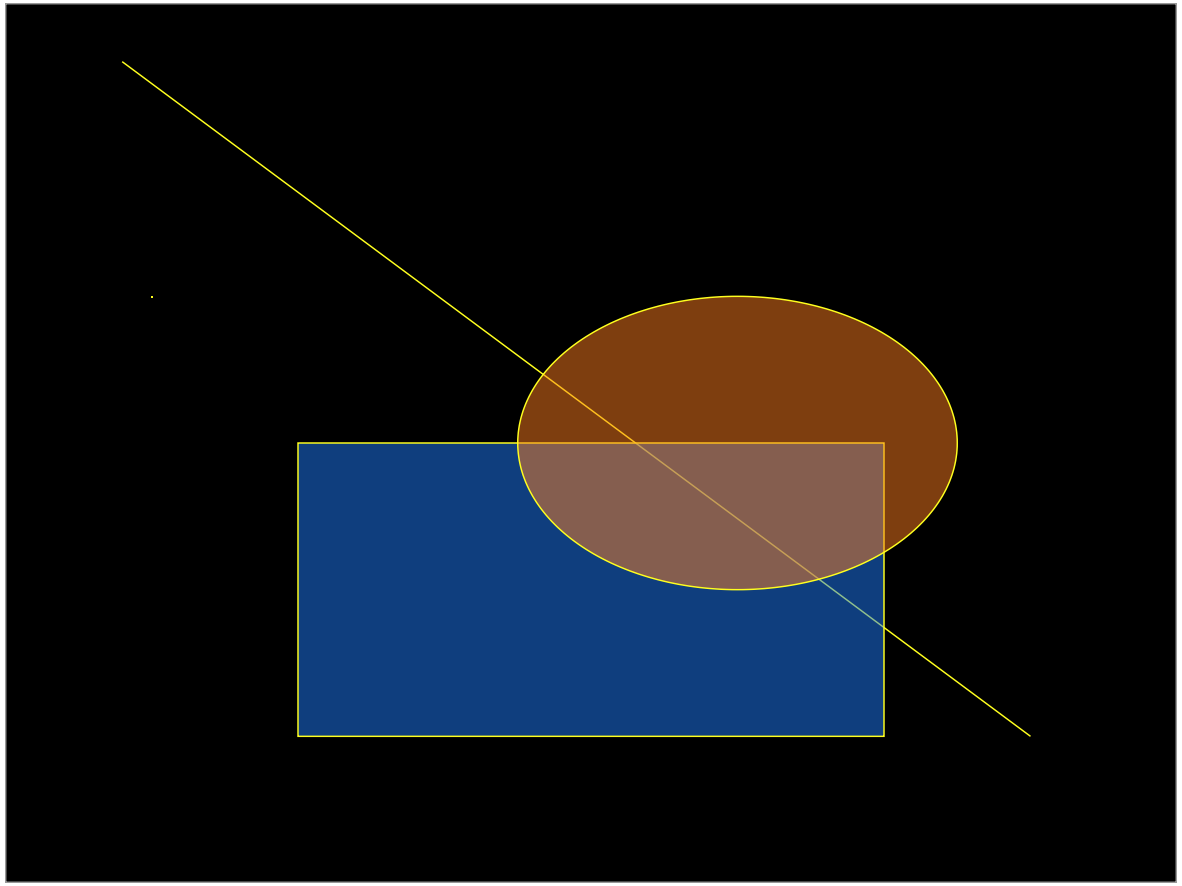
- 複数の色を塗り分ける

```
function setup() {
  createCanvas(800, 600);
}

function draw() {
  background(0); // 背景色
  stroke(255, 255, 31); // 線の色
  point(100, 200); // 点を描画
  line(80, 40, 700, 500); // 線を描画
  fill(31, 127, 255, 127); // 四角形の色
  rect(200, 300, 400, 200); // 四角形を描画
  fill(255, 127, 31, 127);  // 楕円の色
  ellipse(500, 300, 300, 200); // 楕円を描画
}
```

色の塗り分け

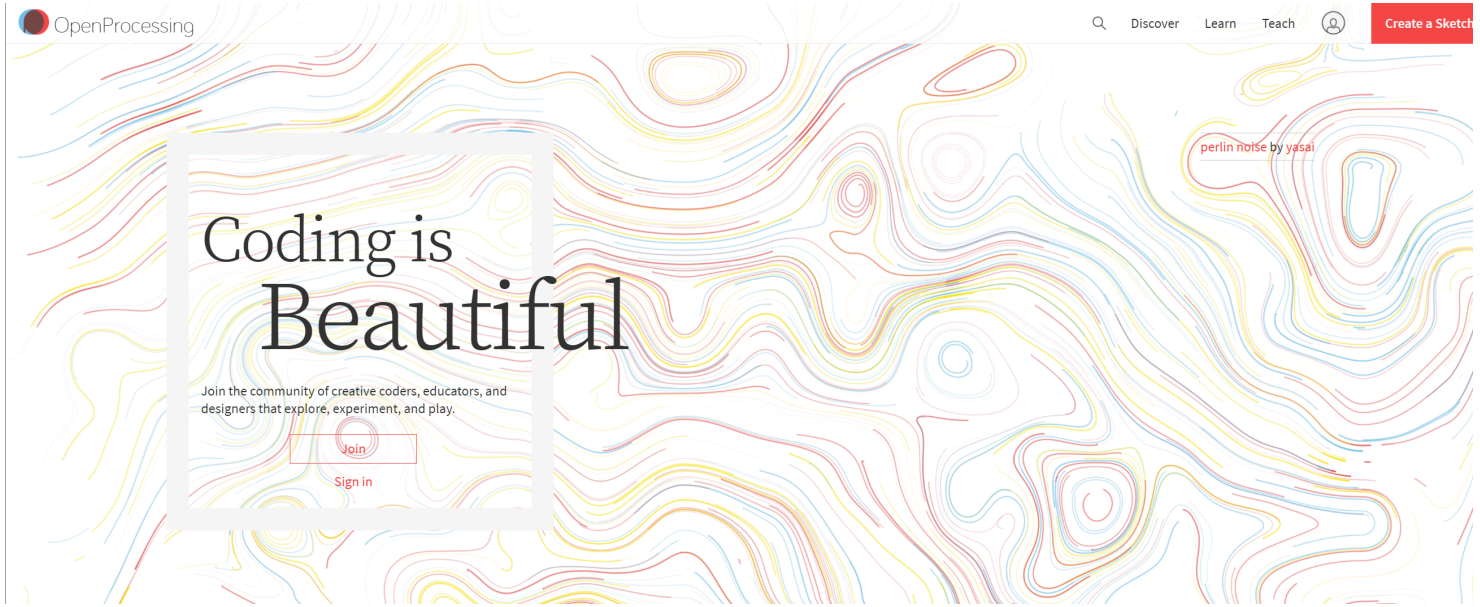
- 色を塗り分ける



OpenProcessingにアカウントを作成して
コードをシェア

OpenProcessingにアカウントを作成してコードをシェア

- OpenProcessing - p5.jsのコードをシェアできるサービス
- https://www.openprocessing.org/
- 無料で利用可能 (有料オプションもあり)
- 早速登録してみましょう! トップページの「Join」ボタンから



OpenProcessingにアカウントを作成してコードをシェア

- 作成したコードをシェアする手順は動画で紹介します!
- とっても簡単!!

本日の課題：p5.js基本図形と色による画面構成

本日の課題：p5.js基本図形と色による画面構成

- テーマ: 「p5.js基本図形と色による画面構成」
  - ここまで解説したp5.jsの機能を活用して画面構成をしてみましょう
  - 完成したら何かタイトルをつける
  - 座標、色、数値による図形の描画に慣れるのが目的です
- 完成した作品はOpenProcessingに投稿してURLを提出
- 次回の授業の冒頭で簡単に作品をいくつか紹介します

かたちとコード

かたちとコード

- コード（プログラミング）で形を描く意味
- どのような表現に向いているのか?

かたちとコード

- **正確**：指定した通りに寸分違わず形を描くことができる
- **反復**：何度でも同じ操作をくりかえす
- **論理（アルゴリズム）**：形を描くルール自体を書くことができる

コンピューターの登場黎明期から、様々な表現が行われてきた

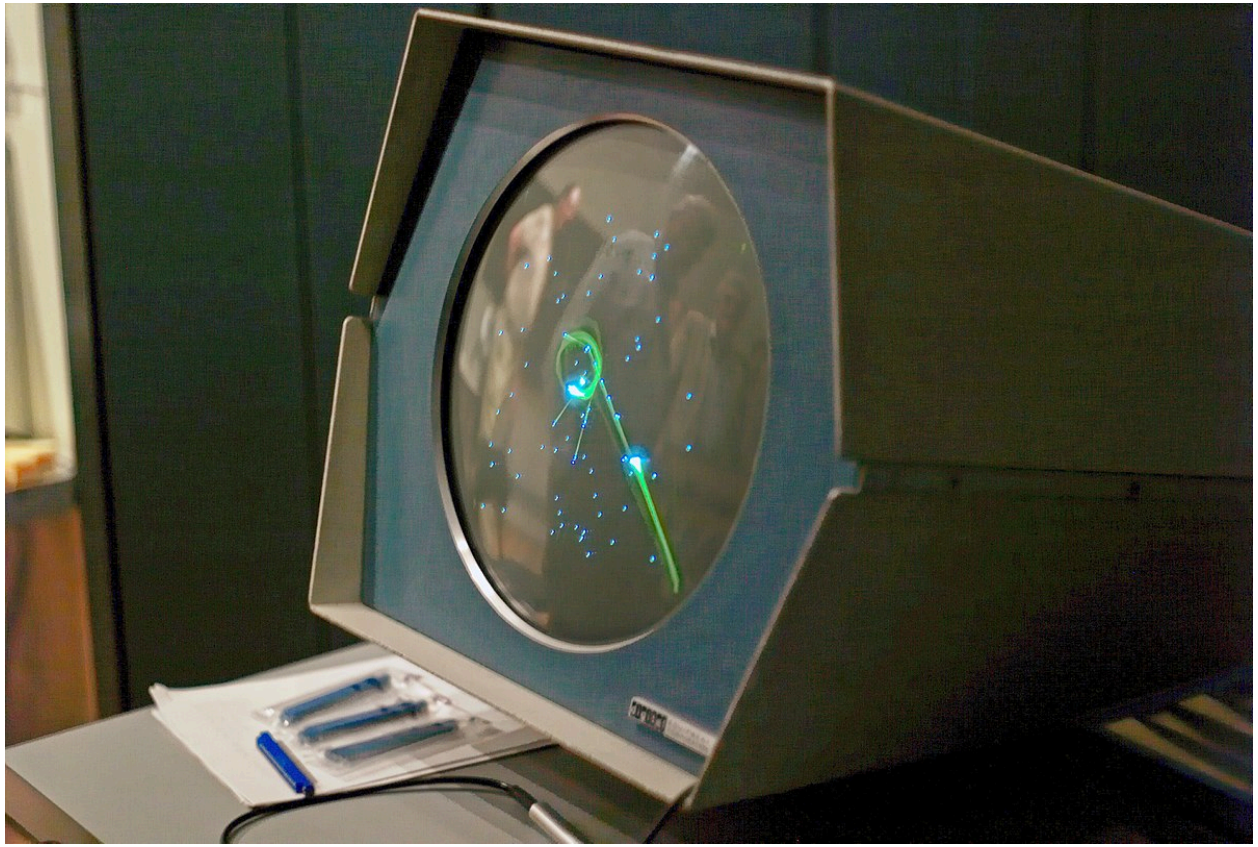
過去へタイムスリップ
1960〜70年代のコンピュータ・アート

1960〜70年代のコンピュータ・アート

- ポイント: 当時のプログラミング環境
  - 大学や研究所が中心
  - 限られた一部のエンジニア、プログラマーのみが触れることが可能
- そのような時代のコンピュータ・アートは、どのようなものだったのか?

1960〜70年代のコンピュータ・アート

- 最初期のコンピュータゲーム “Spacewar!” (1962)
- https://youtu.be/1EWQYAfuMYw?t=893



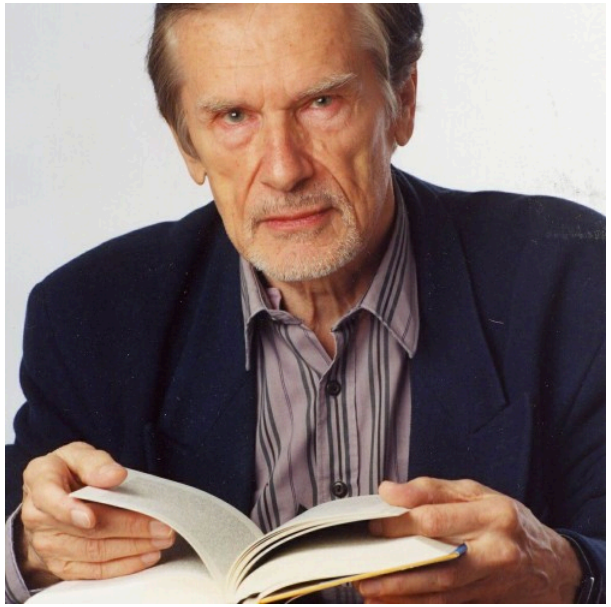
1960〜70年代のコンピュータ・アート

- 実際にプレイすることも可能!
- https://www.masswerk.at/spacewar/



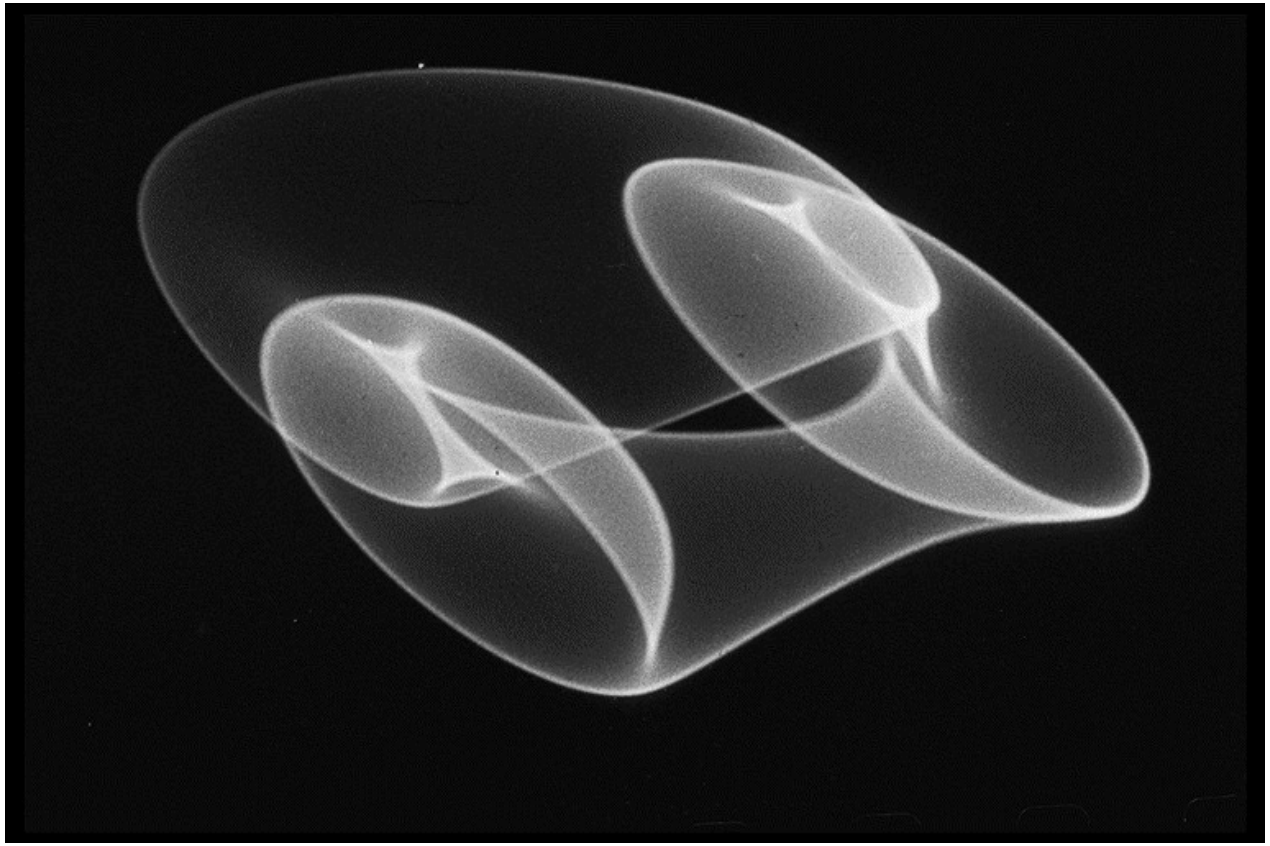
1960〜70年代のコンピュータ・アート

- Herbert Franke
- 1927年、ウィーン生まれ
- 1950年代から、アナログコンピュータとオシロスコープによる作品を制作
- コンピュータアートのパイオニア



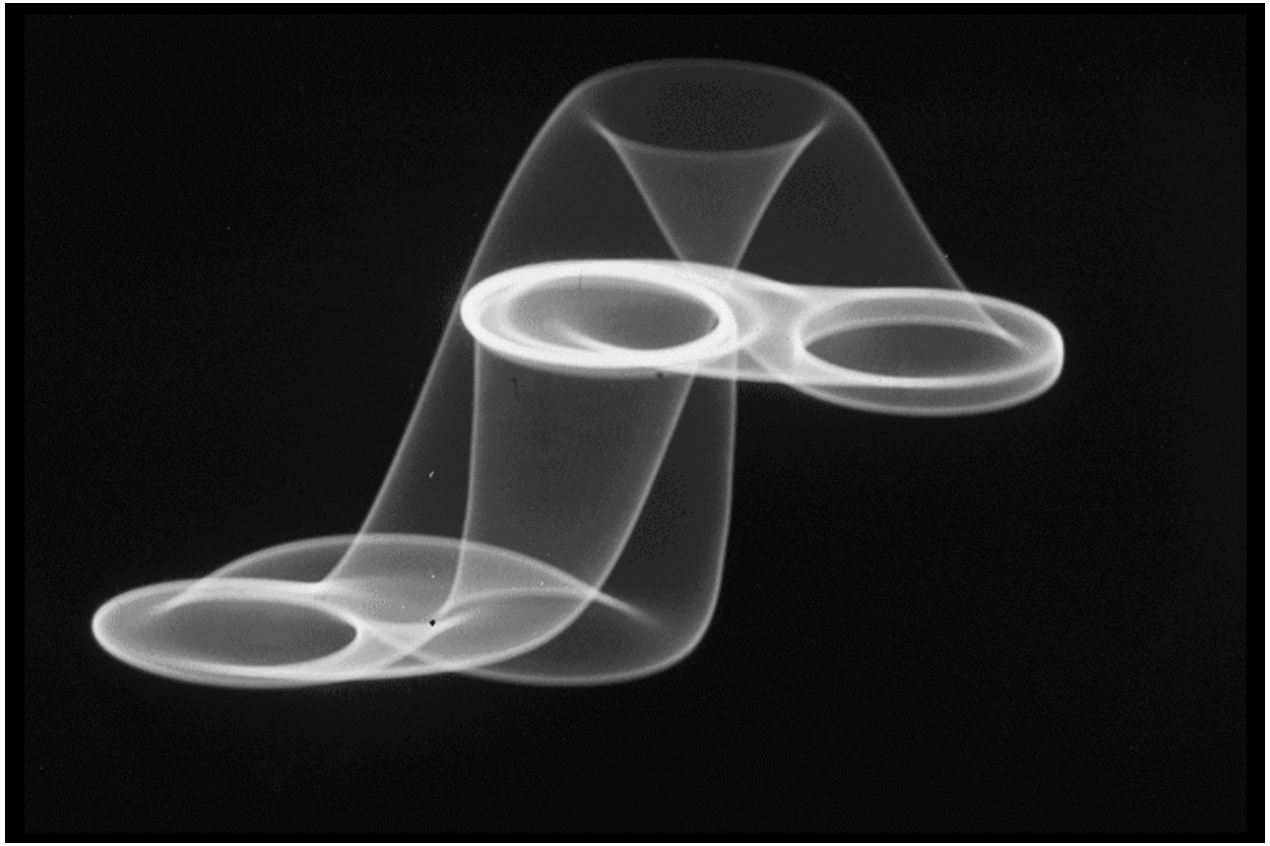
1960〜70年代のコンピュータ・アート

- Herbert Franke, Electronic Graphics, 1961-62
- アナログコンピュータとオシロスコープによる作品



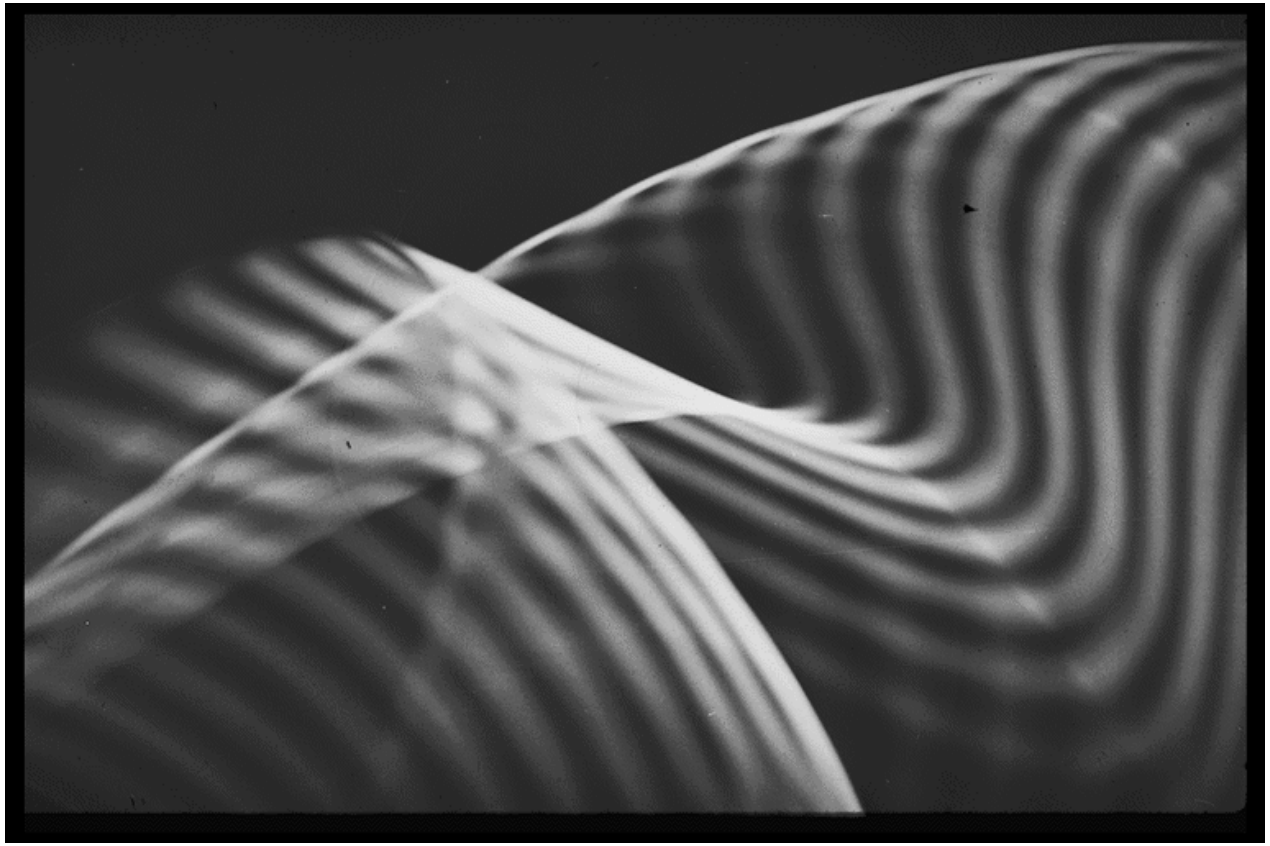
1960〜70年代のコンピュータ・アート

- Herbert Franke, Electronic Graphics, 1961-62
- アナログコンピュータとオシロスコープによる作品



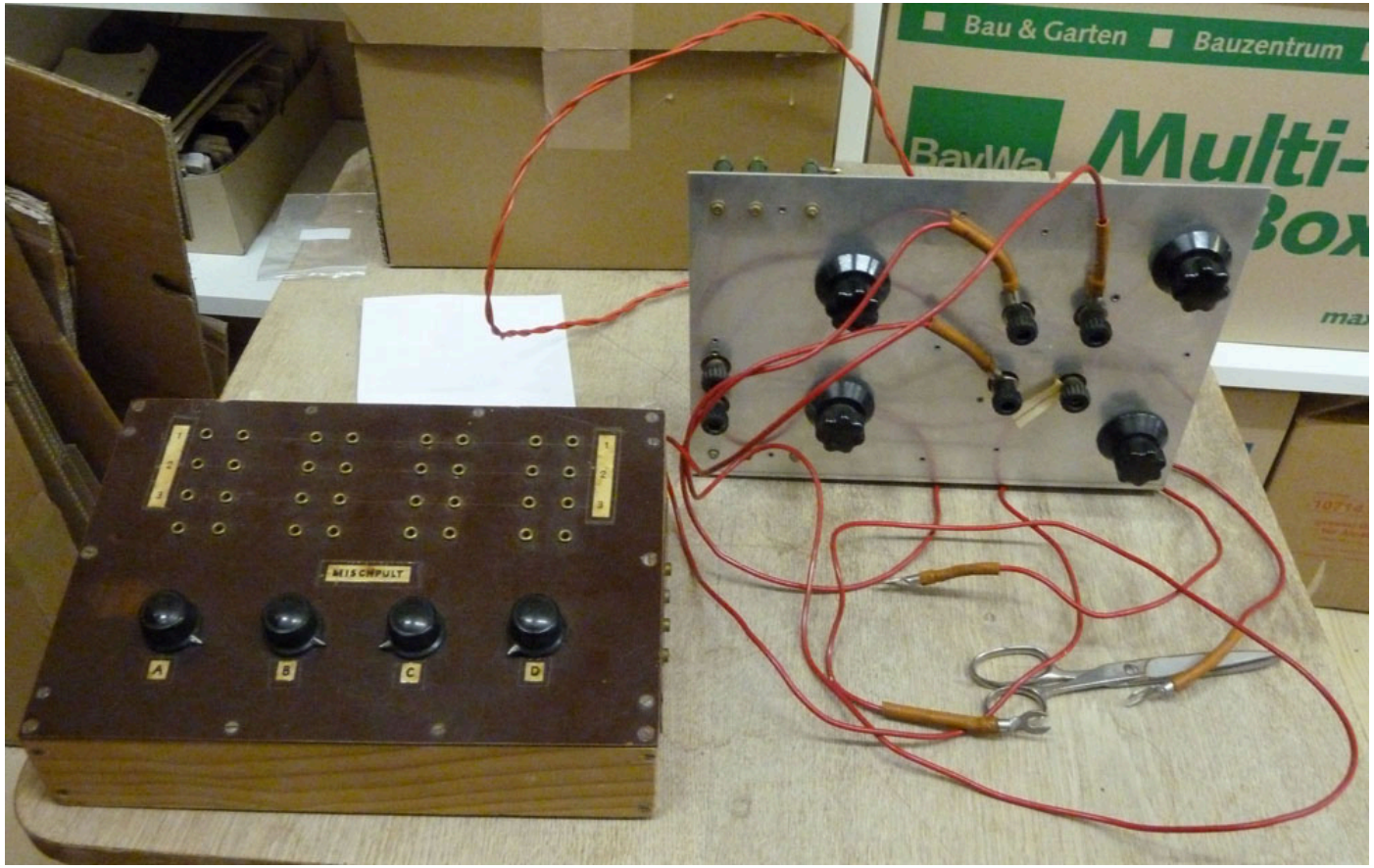
1960〜70年代のコンピュータ・アート

- Herbert Franke, Electronic Graphics, 1961-62
- アナログコンピュータとオシロスコープによる作品



1960〜70年代のコンピュータ・アート

- Herbert Franke のアナログ装置



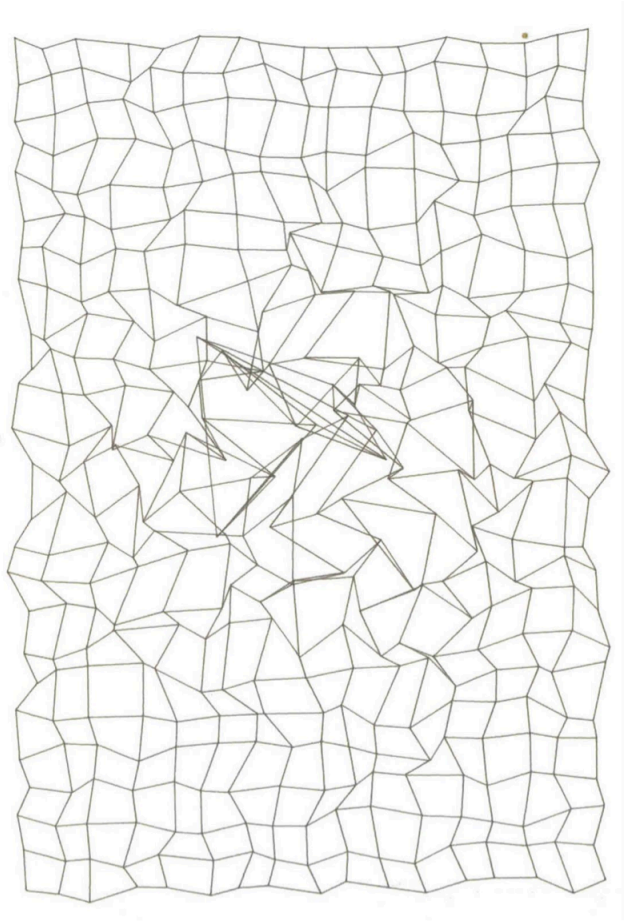
1960〜70年代のコンピュータ・アート

- Georg Nees
- 1926年、ニュルンベルク生まれ
- 1951年からシーメンス社に在籍
- デジタルコンピュータをプログラミングして、図形生成を行ったパイオニア



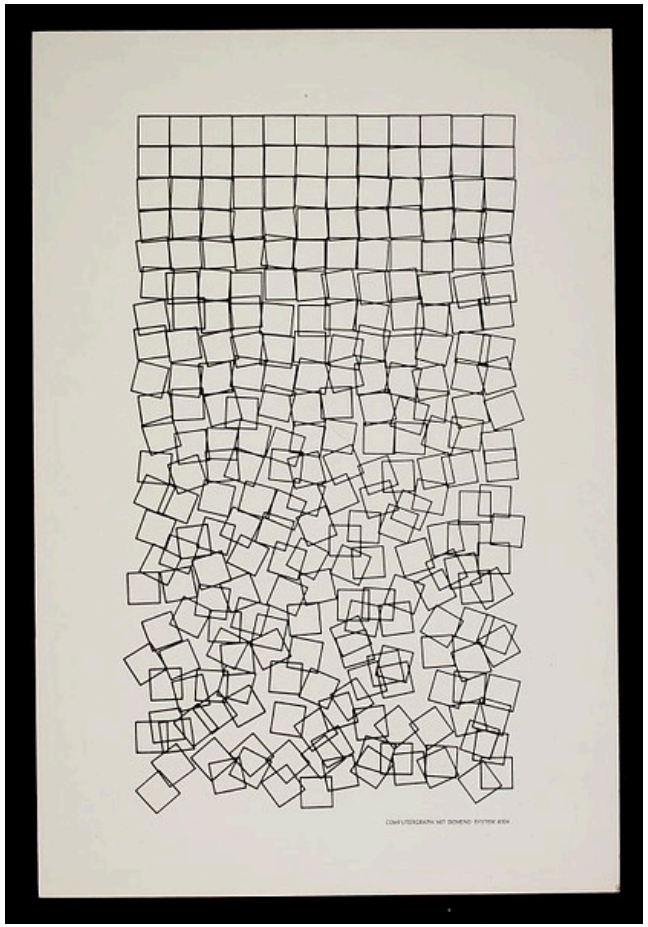
1960〜70年代のコンピュータ・アート

- Georg Nees, 1965-1968.



1960〜70年代のコンピュータ・アート

- Georg Nees, Schotter 1968.

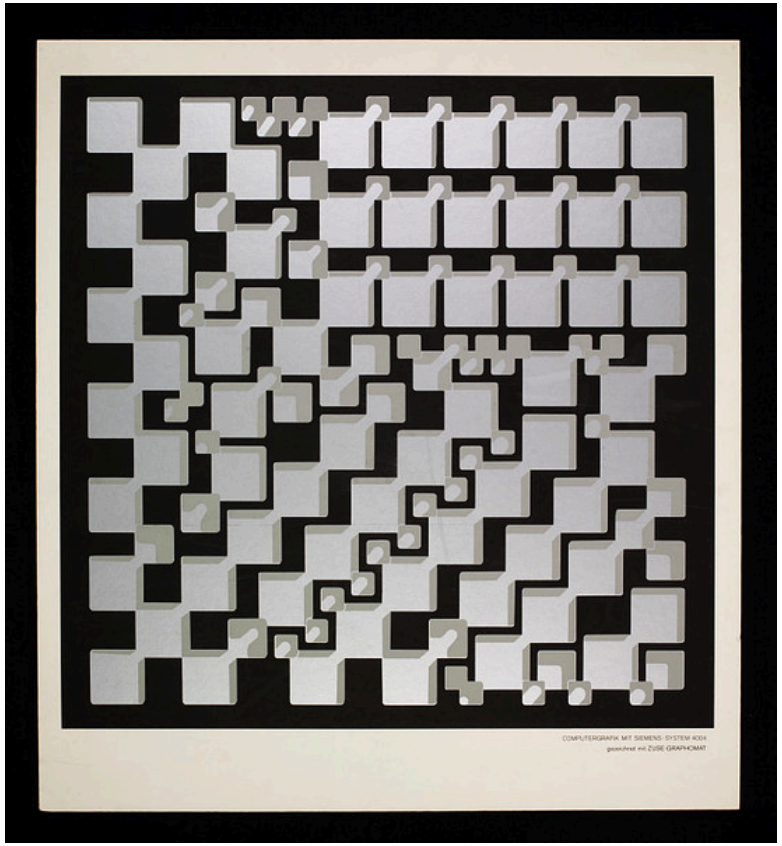


```
'BEGIN''COMMENT'SCHOTTER.,
'REAL'R,PIHALB,PI4T.,
'INTEGER'I.,
'PROCEDURE'QUAD., # generates the main figure, a square
'BEGIN'
'REALP1,Q1PSI.,'INTEGER'S.,
JE1.=5*I/264.,JA1.=-JE1., # random generator 1
JE2.=PI4T*(1+I/264)., # random generator 2
JA2.=PI4T*(1-I/264).,
P1.=P+5+J1.,Q1.=Q+5+J1.,# changing the coordinates
PSI.=J2.,
LEER(P1+R*COS(PSI),
Q1+R*SIN(PSI)).,
'FOR'S.=1'STEP'1'UNTIL'4'DO'
'BEGIN'PSI.=PSI+PIHALB.,
LINE(P1+R*COS(PSI),Q1+R*SIN(PSI))
'END'.,I.=I+1
'END'QUAD.,
R.=5*1.4142.,
PIHALB.=3.14159*.5.,PI4T.=PIHALB*.5.,
I.=0.,
SERIE(10.0,10.0,22,12,QUAD) # multiply the main figure
'END'SCHOTTER.,
```

Nees, Georg: Generative Computergraphik. 1969, p. 241

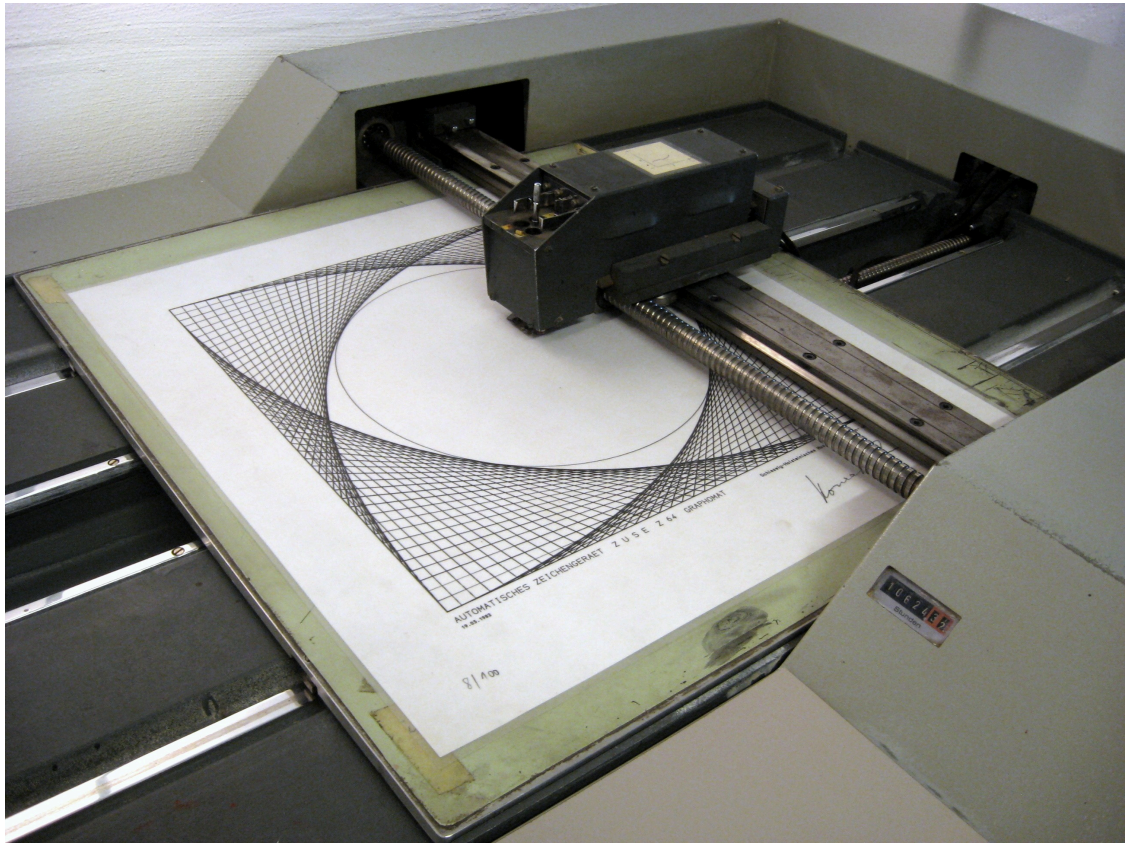
1960〜70年代のコンピュータ・アート

- Georg Nees, ‘Plastik 1’, 1965-8
- アルミレリーフ、工作機械をプログラムで制御



1960〜70年代のコンピュータ・アート

- Georg Nees が使用していたプロッタ
- ZUSE Z64



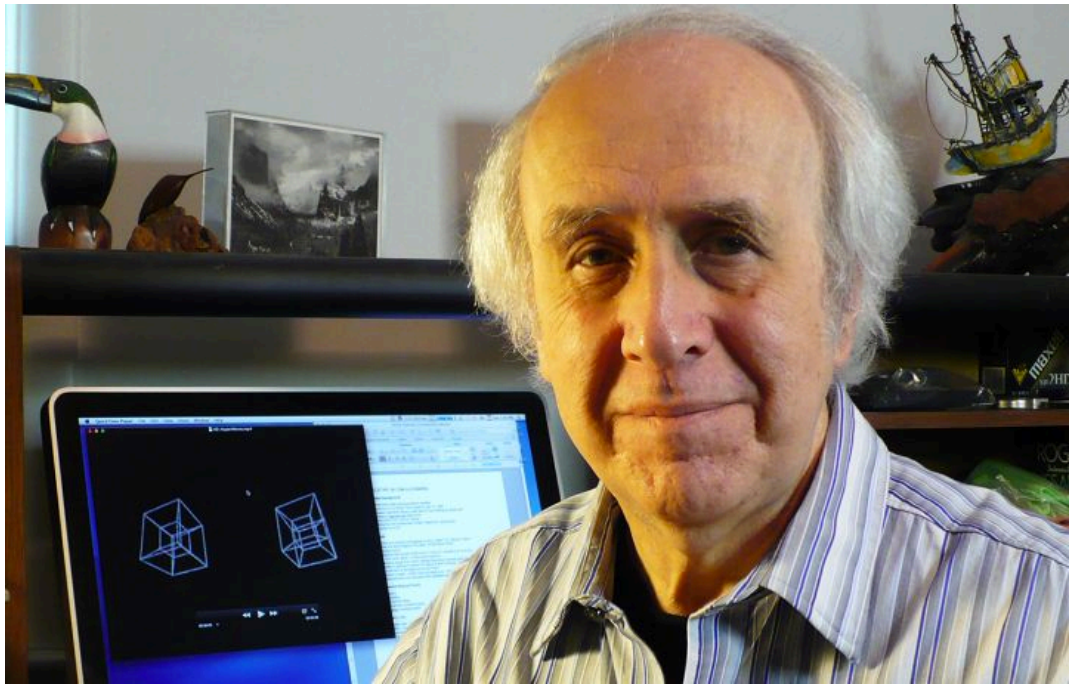
1960〜70年代のコンピュータ・アート

- Georg Nees が使用していた頃のコンピュータ
- Siemens 2002



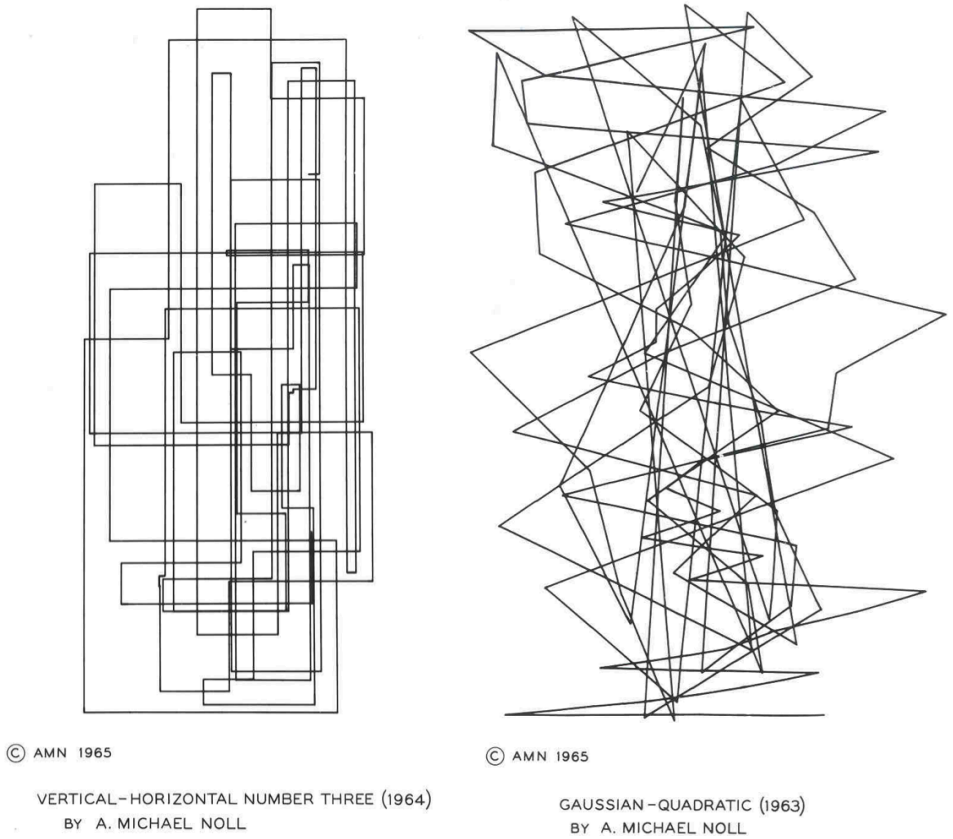
1960〜70年代のコンピュータ・アート

- A. Michael Noll
- 1939年生まれ
- 1961年から15年間、AT&Tベル研究所で基礎研究にとりくむ



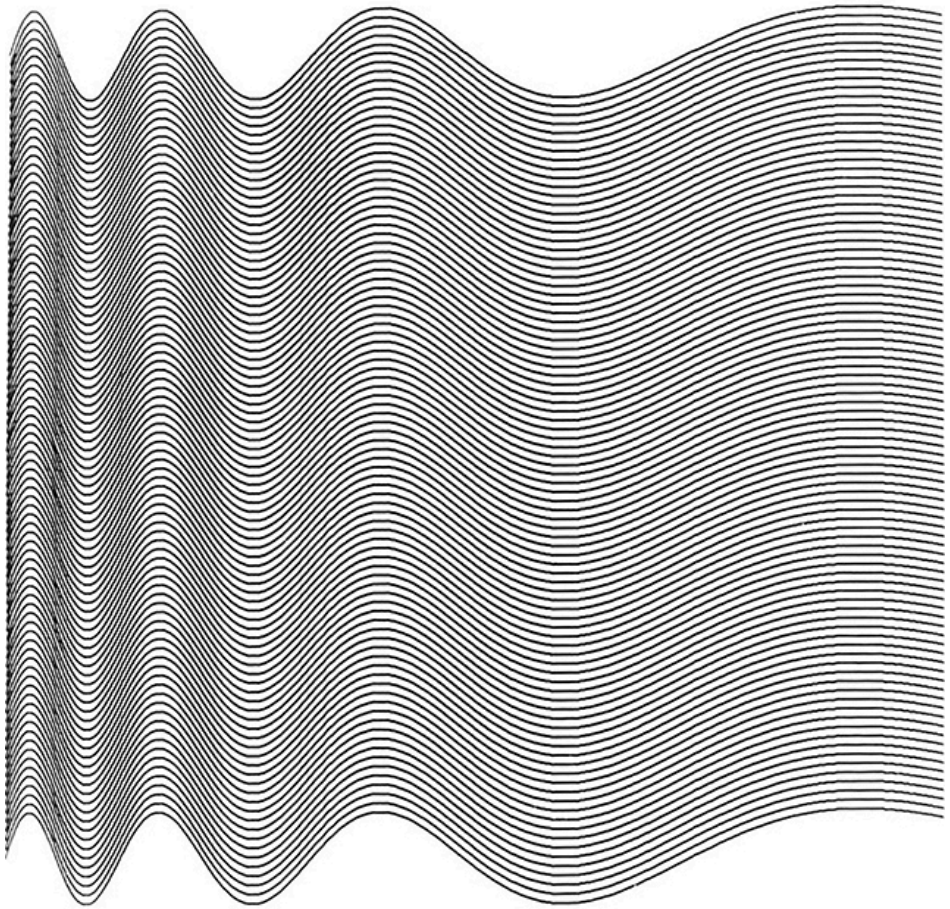
1960〜70年代のコンピュータ・アート

- A. Michael Noll, “Vertical-Horizontal No. 3”, 1964
- “Gaussian Quadratic” 1963



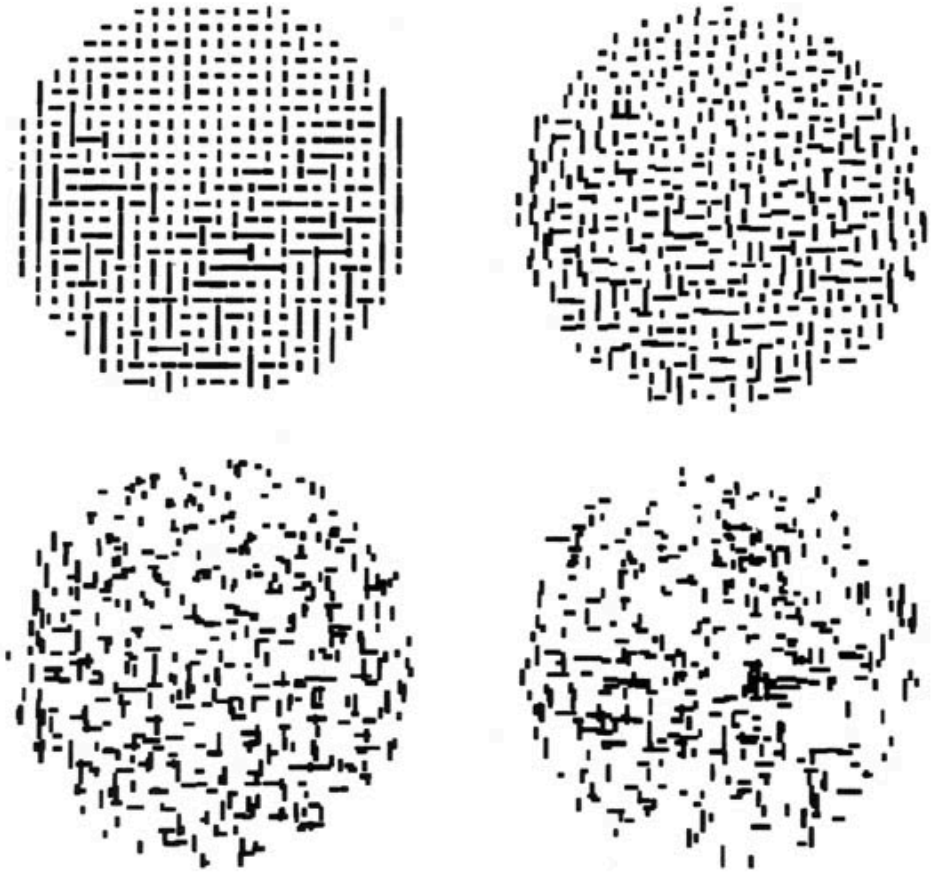
1960〜70年代のコンピュータ・アート

- A. Michael Noll, “Ninety Parallel Sinusoids With Linearly Increasing Period”, 1964



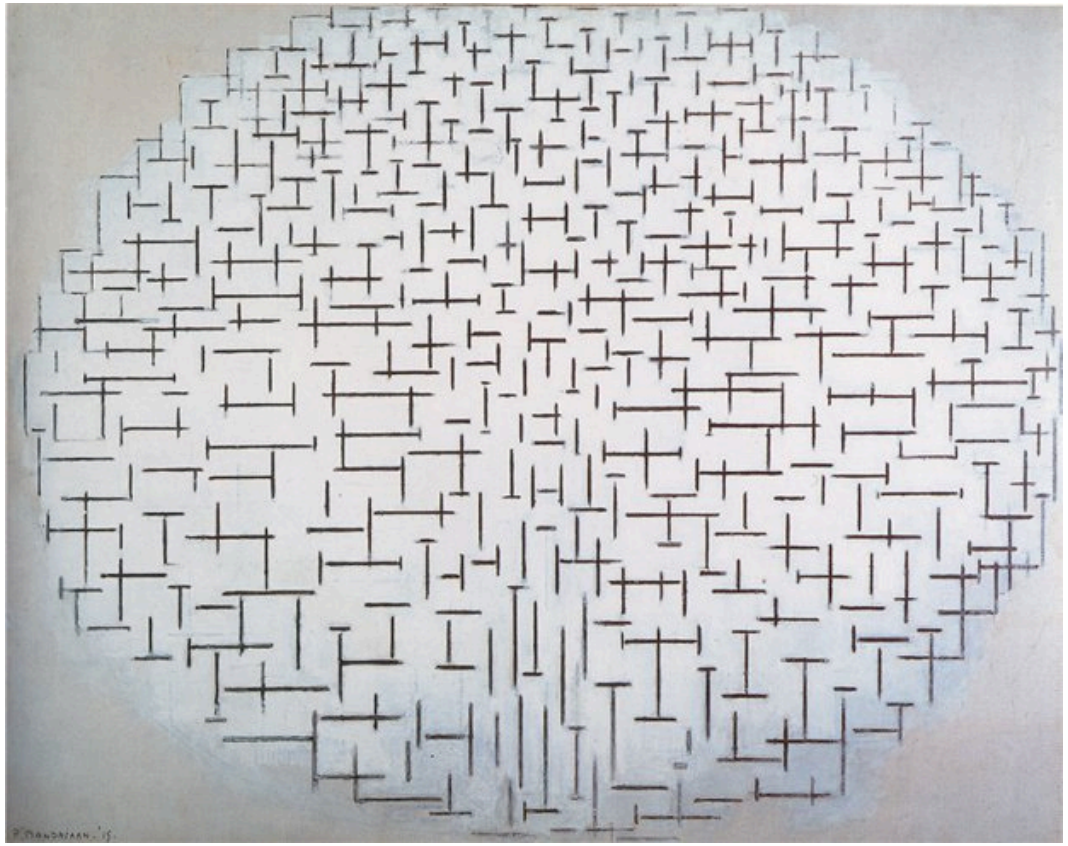
1960〜70年代のコンピュータ・アート

- A. Michael Noll, “Four computer-generated random patterns”, 1965



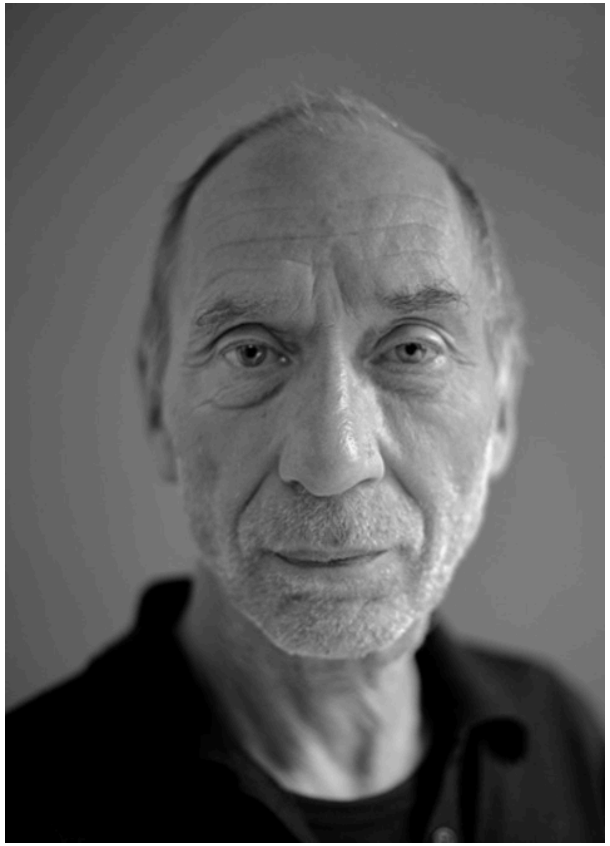
1960〜70年代のコンピュータ・アート

- A. Michael Noll, “Four computer-generated random patterns”, 1965
- モンドリアンの「線によるコンポジション (1917)」をベースにしている



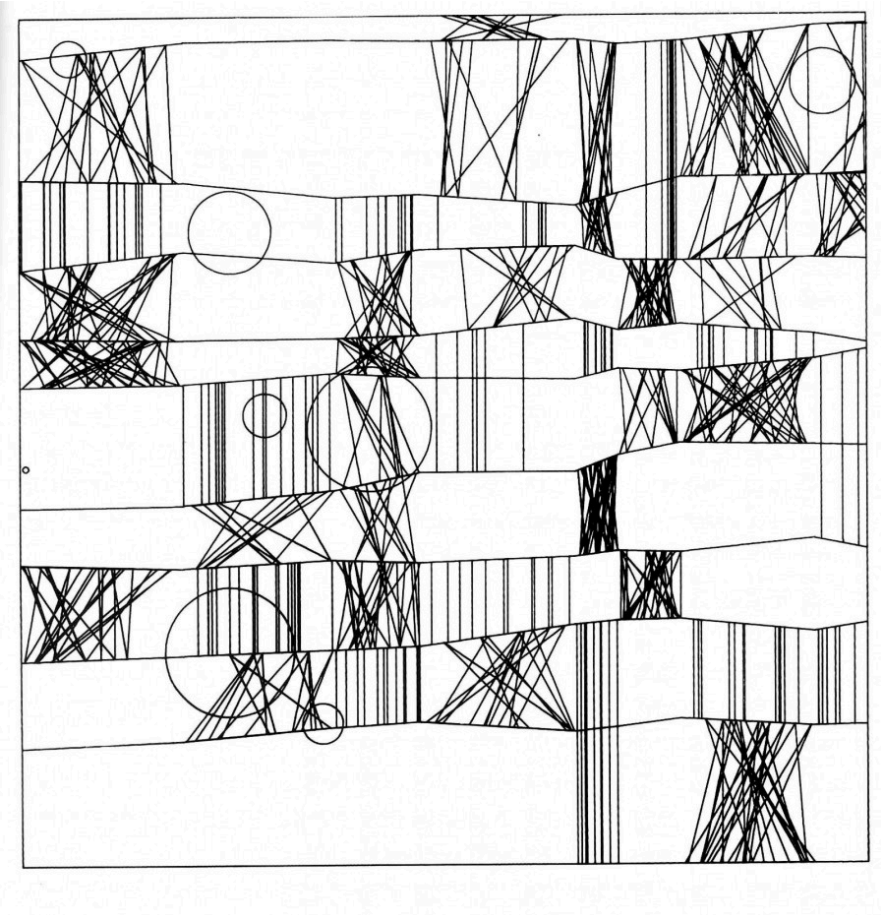
1960〜70年代のコンピュータ・アート

- Frieder Nake
- 1938年、ドイツ、シュトゥットガルト生まれ



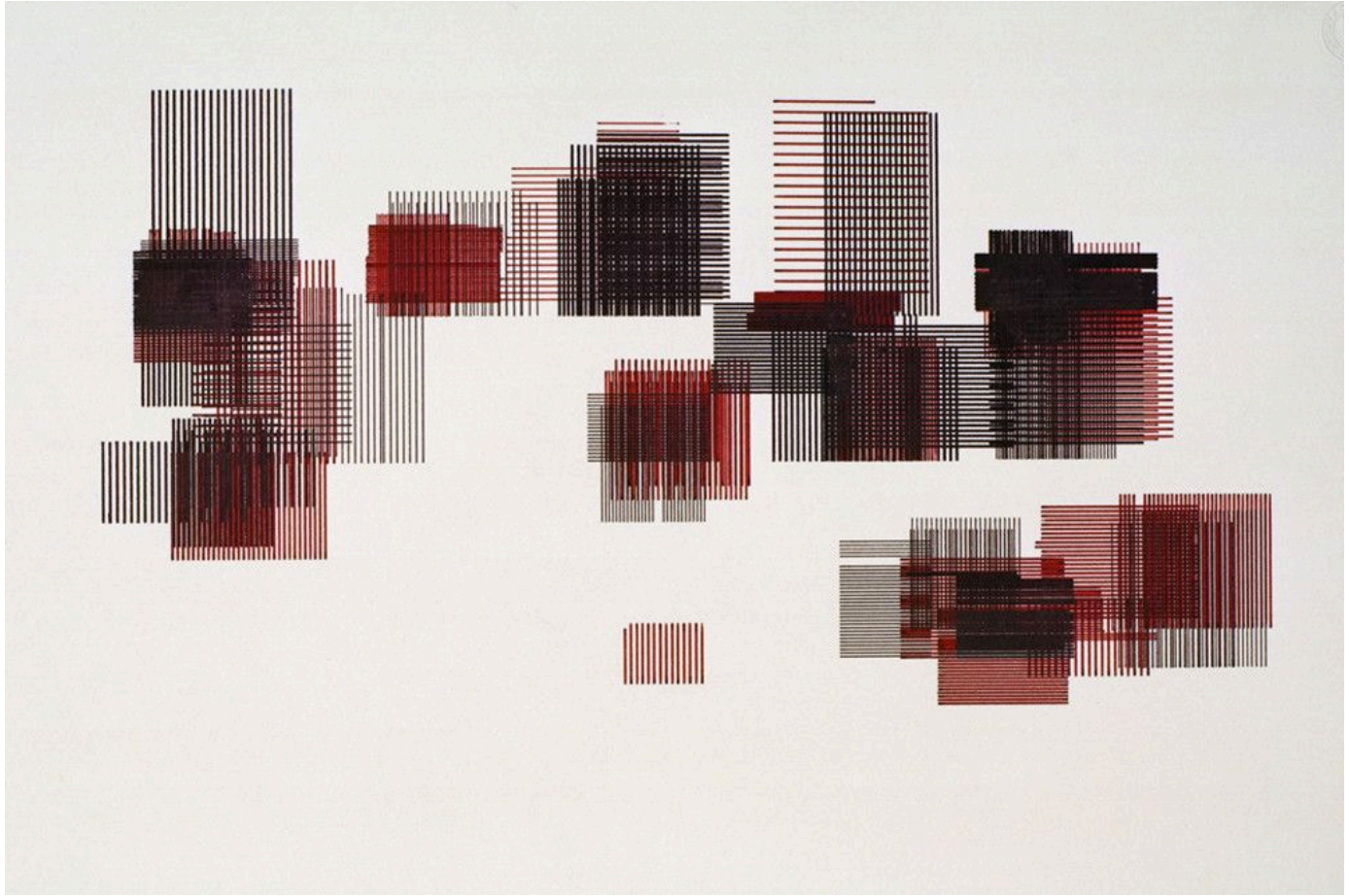
1960〜70年代のコンピュータ・アート

- Frieder Nake, 13/9/65 Nr. 2, 1965



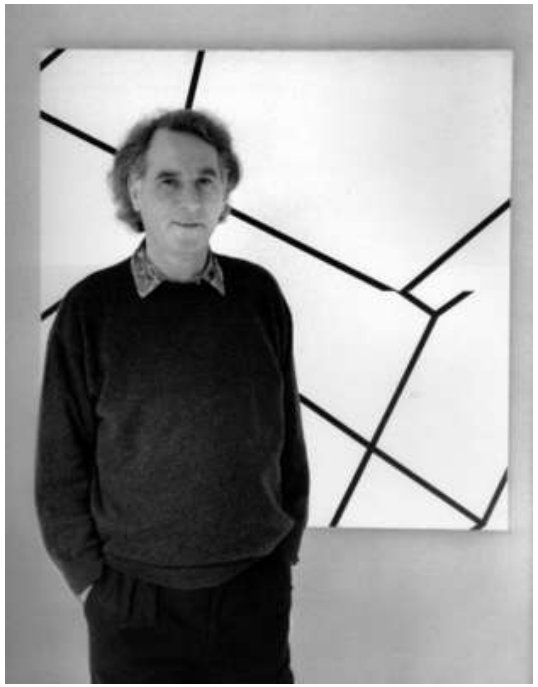
1960〜70年代のコンピュータ・アート

- Frieder Nake, 1965



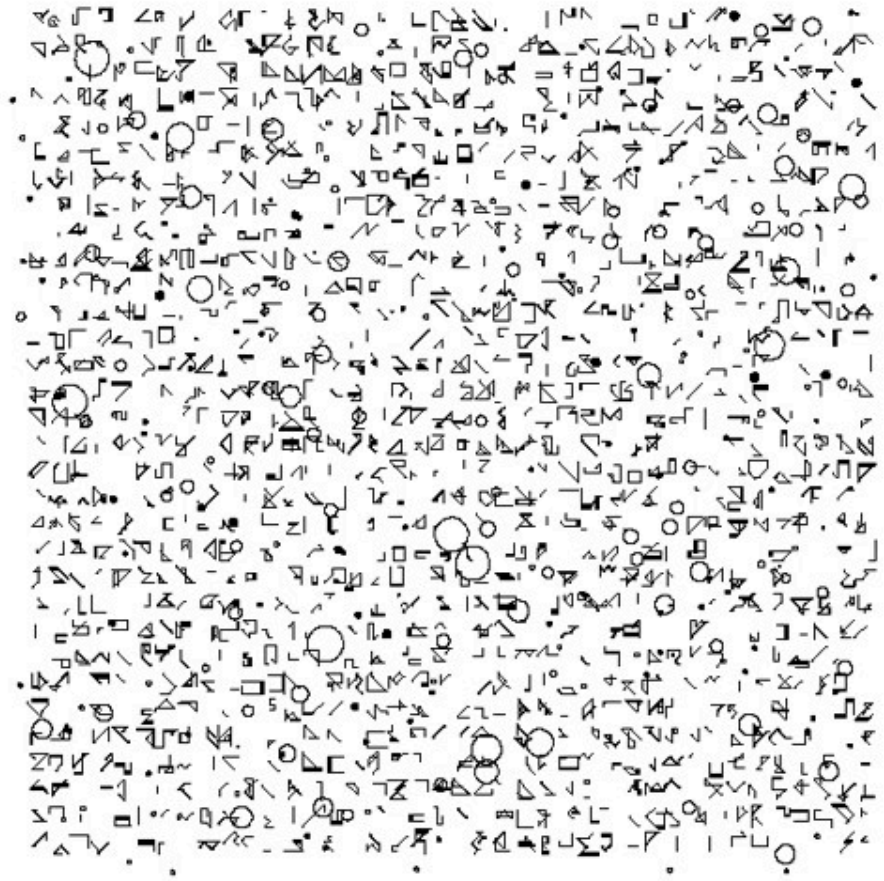
1960〜70年代のコンピュータ・アート

- Manfred Mohr
- 1938年ドイツ生まれ
- 1963年からパリ、1981年からニューヨークに在住して制作



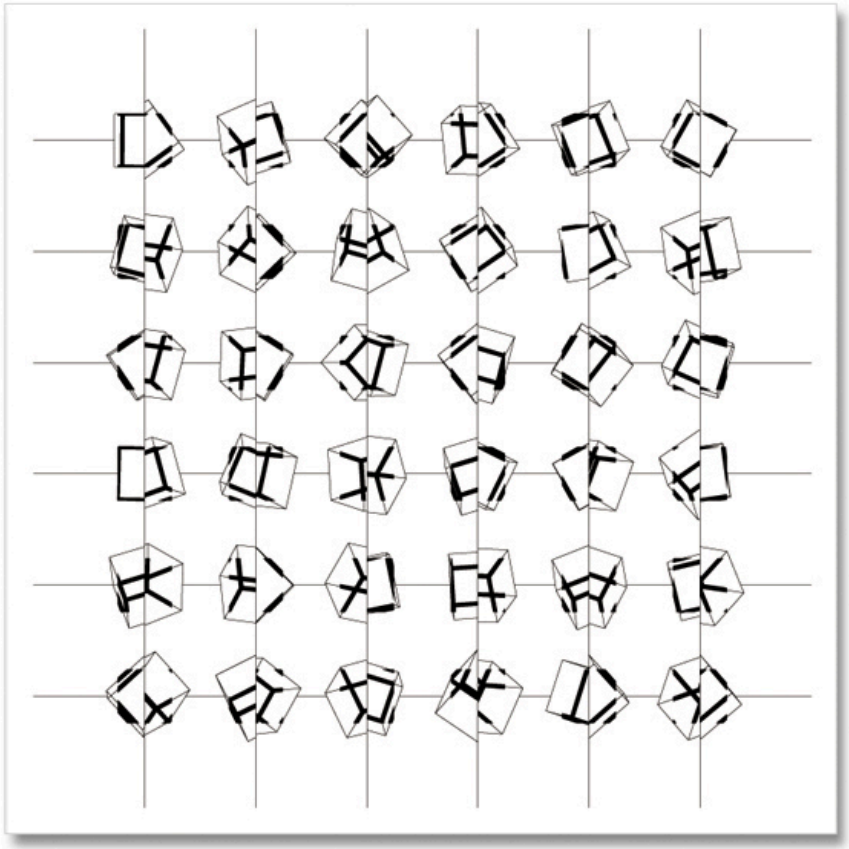
1960〜70年代のコンピュータ・アート

- Manfred Mohr, “a formal language”, 1970



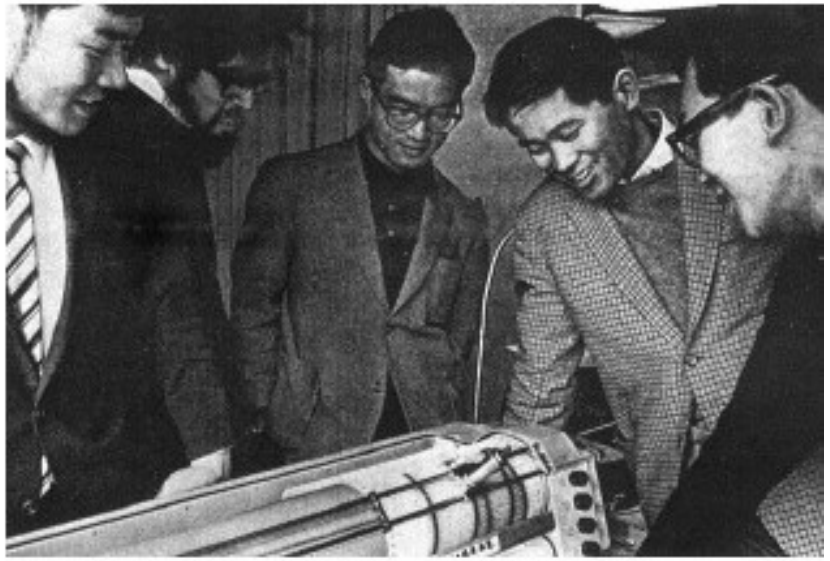
1960〜70年代のコンピュータ・アート

- Manfred Mohr, “Cubic Limit” 1972-77



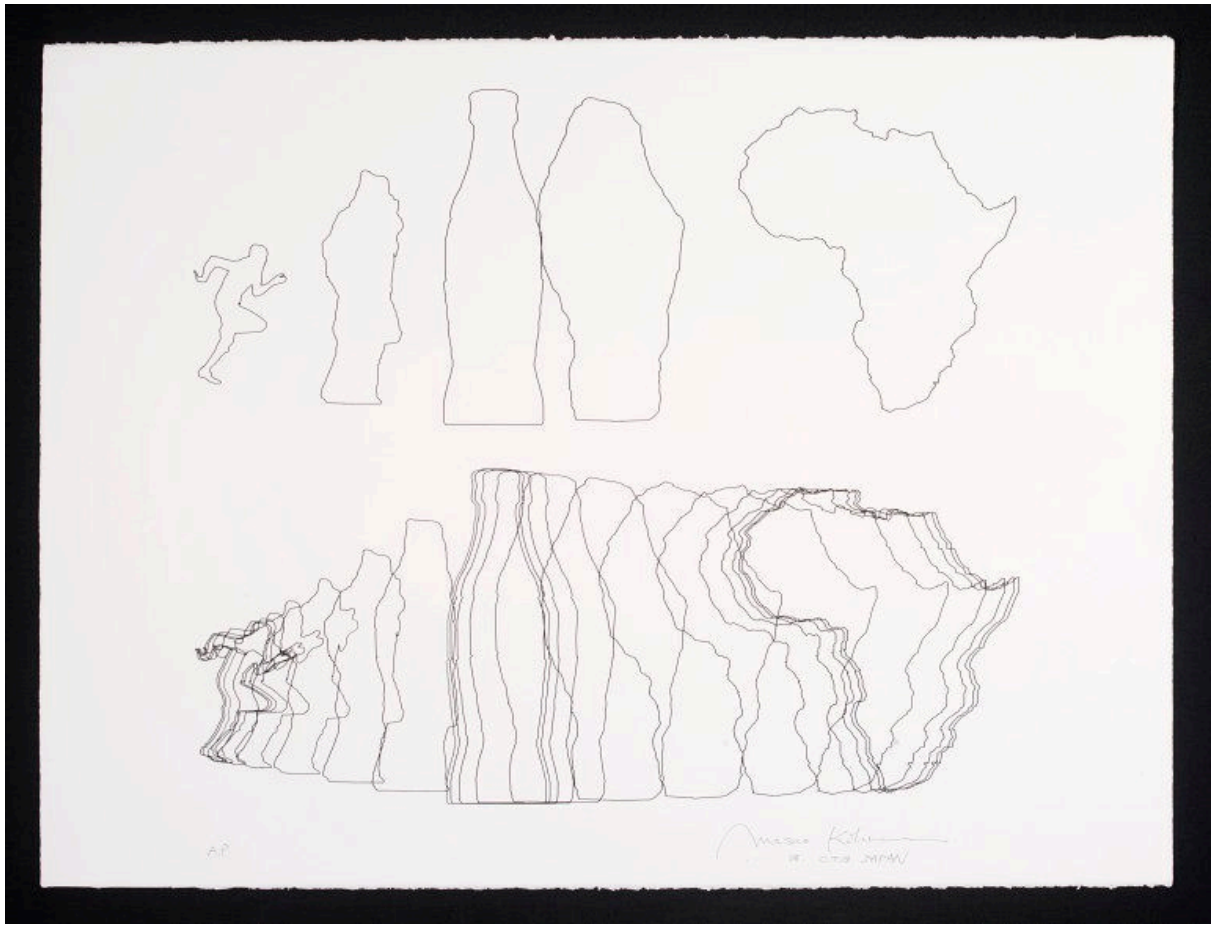
1960〜70年代のコンピュータ・アート

- Computer Technique Group (CTG)
- 槌屋治紀と幸村真佐男の出会いをきっかけに、1966年12月に山中邦夫と柿崎純一郎を加えた４人（最終的に10名）で結成されたコンピュータ・アート集団



1960〜70年代のコンピュータ・アート

- Computer Technique Group, Running Cola is Africa, 1969



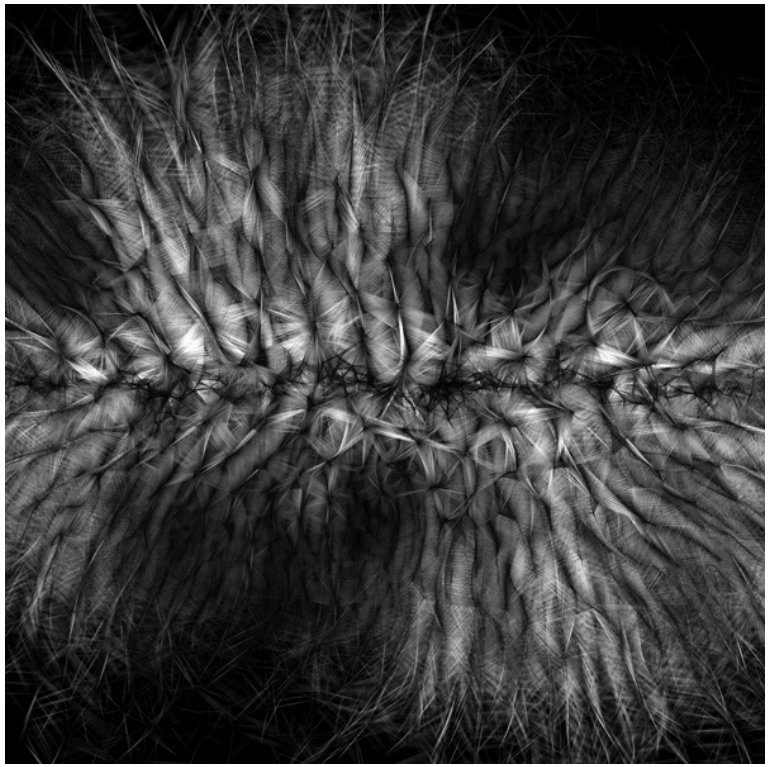
1960〜70年代のコンピュータ・アート

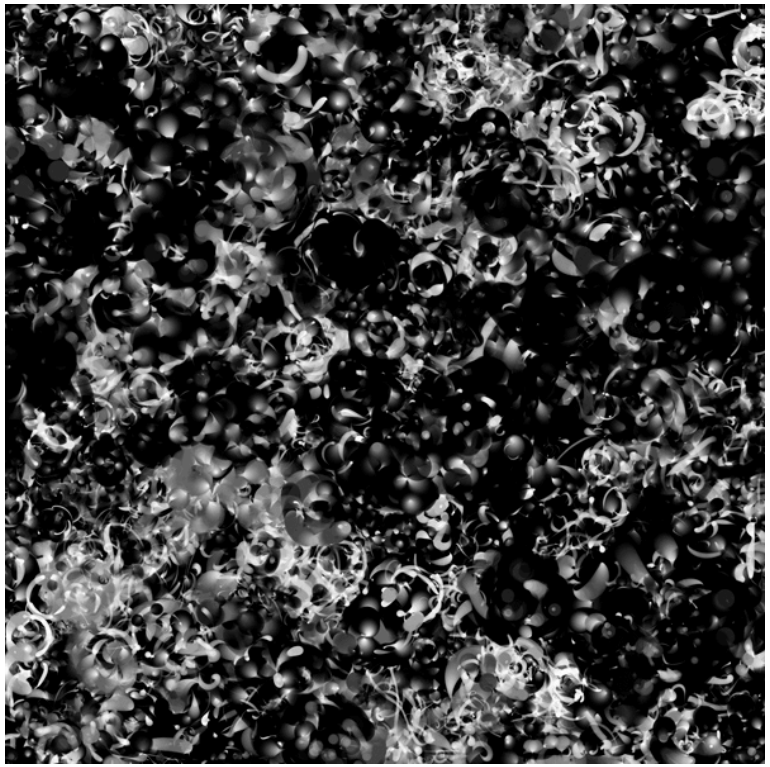
- Computer Technique Group, Shot Kennedy, 1967



かたちとコード

- 現代でも、コードによる美学の探求は進められている
- Casey Reas : Process Compendium 2004-2010 https://vimeo.com/22955812





コードと形

- 現在では、PCさえあれば誰でもコードによる表現ができる時代
- さっそく始めていきましょう!!